

# Different Parts in a Worker Node of



**kubernetes**

# Kubelet

kube-apiserver



Master

Manage, Plan, Schedule, Monitor  
Nodes

kubelet



Worker Nodes

Host Application as Containers

Controller-  
Manager

D  
CLUSTER

XIXIXI

e-scheduler

docker

docker

docker

In a cluster of K8S, everywhere that exists one kubelet component, Kubernetes label it as a node.

# Kubernetes Architecture



## Master

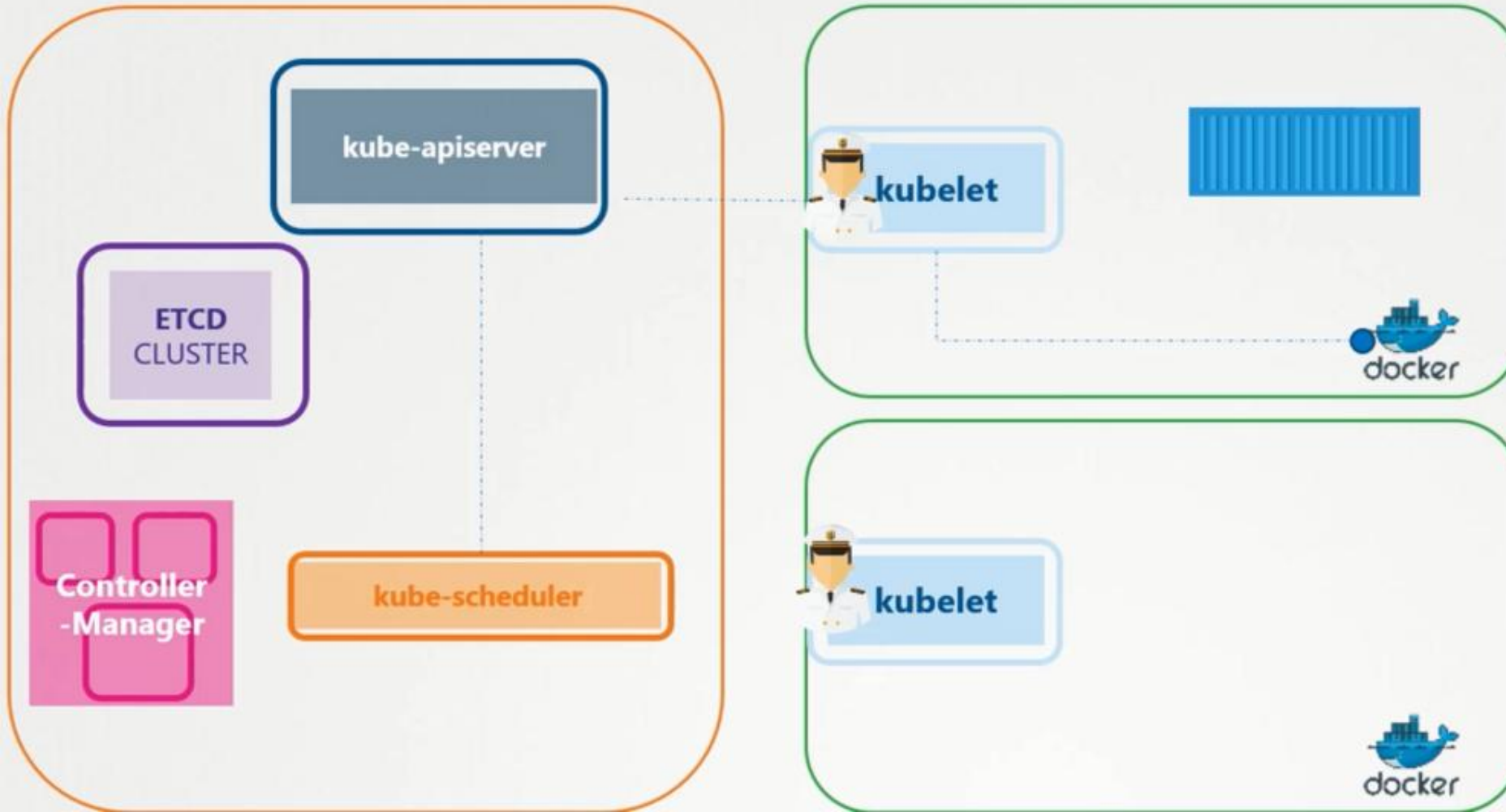
Manage, Plan, Schedule, Monitor  
Nodes



## Worker Nodes

Host Application as Containers

Register Node



1. When kubelet starts, it registers its node with the API server
2. Sends node information (CPU, memory, OS, etc.)

# Kubernetes Architecture



## Master

Manage, Plan, Schedule, Monitor Nodes



## Worker Nodes

Host Application as Containers

kube-apiserver

ETCD  
CLUSTER

Controller  
-Manager

kube-scheduler

kubelet



kubelet



Register Node

Create PODs

1. Watches API server for pods assigned to its node
2. Pulls container images
3. Starts containers via container runtime (Docker/containerd)

# Kubernetes Architecture



## Master

Manage, Plan, Schedule, Monitor Nodes



## Worker Nodes

Host Application as Containers

kube-apiserver

ETCD  
CLUSTER

Controller  
-Manager

kube-scheduler

kubelet



kubelet



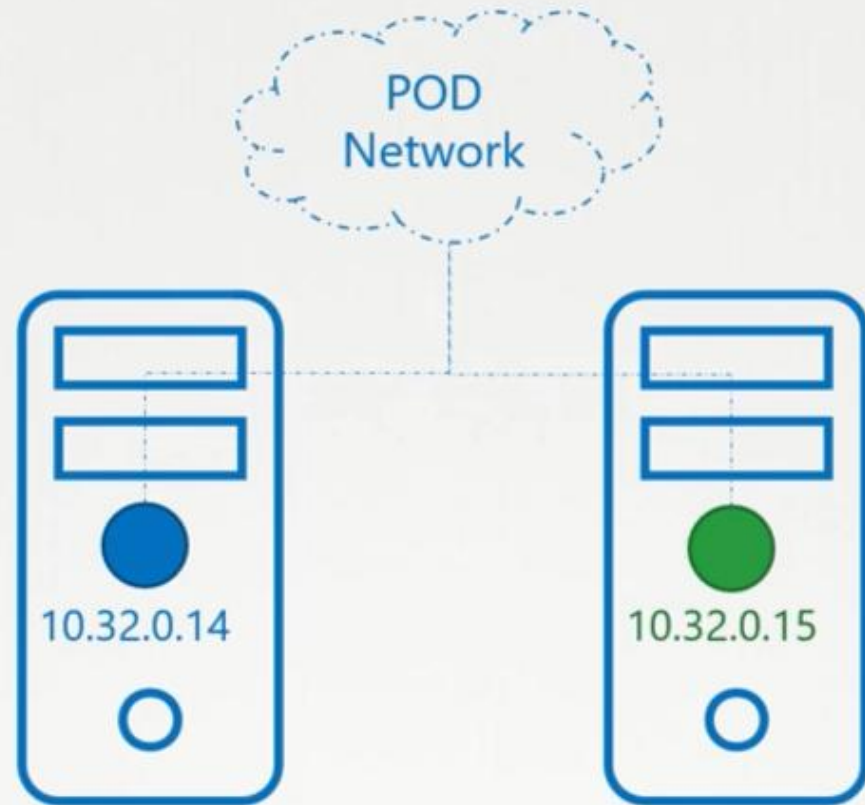
Register Node

Create PODs

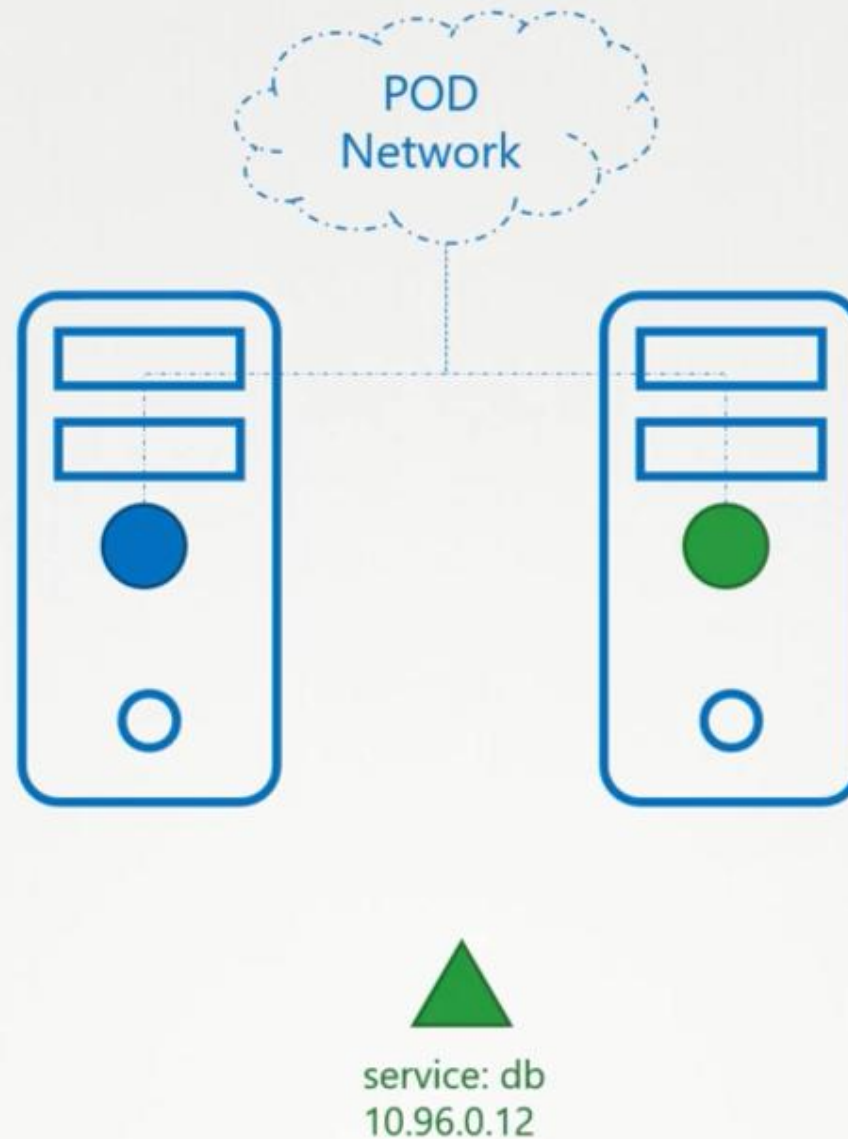
Monitor Node & PODs

1. Sends heartbeats to API server.
2. Reports node status (CPU, memory, disk usage).
3. Monitors pod health, restarts failed containers.
4. Reports pod status back to API server.

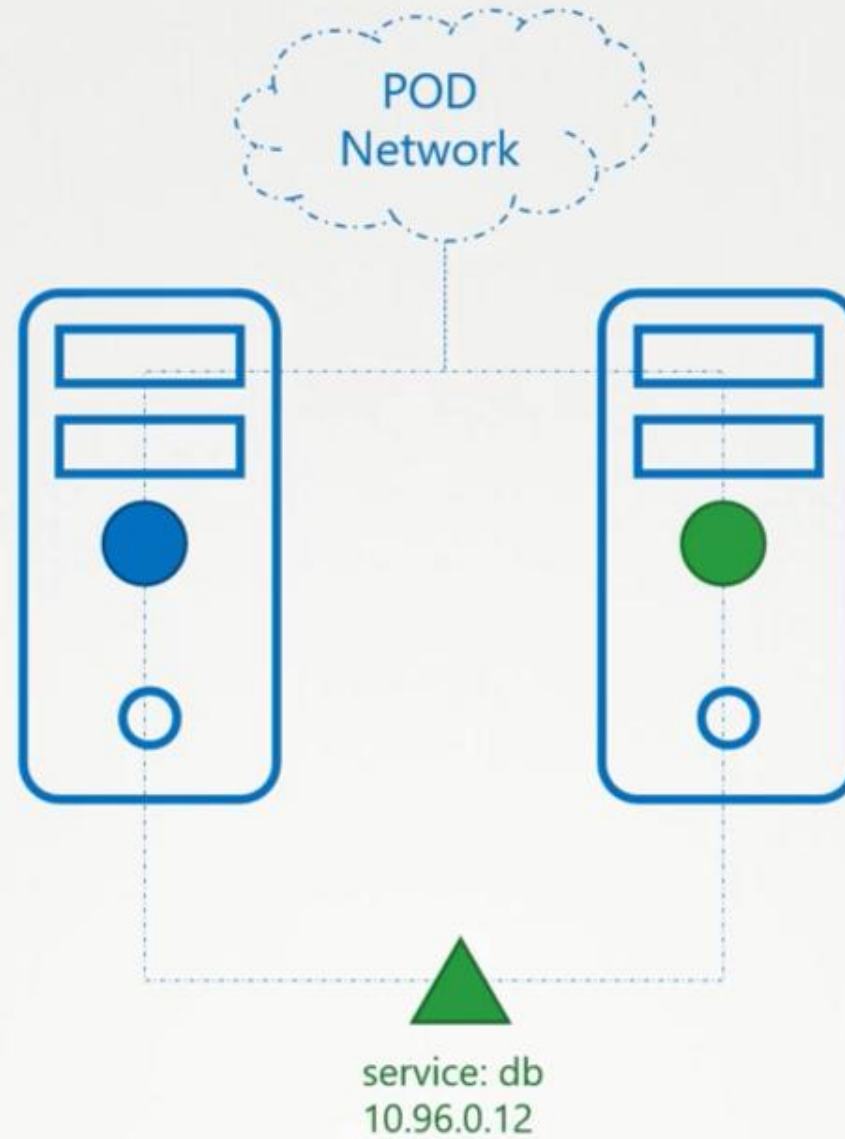
# Kube Proxy



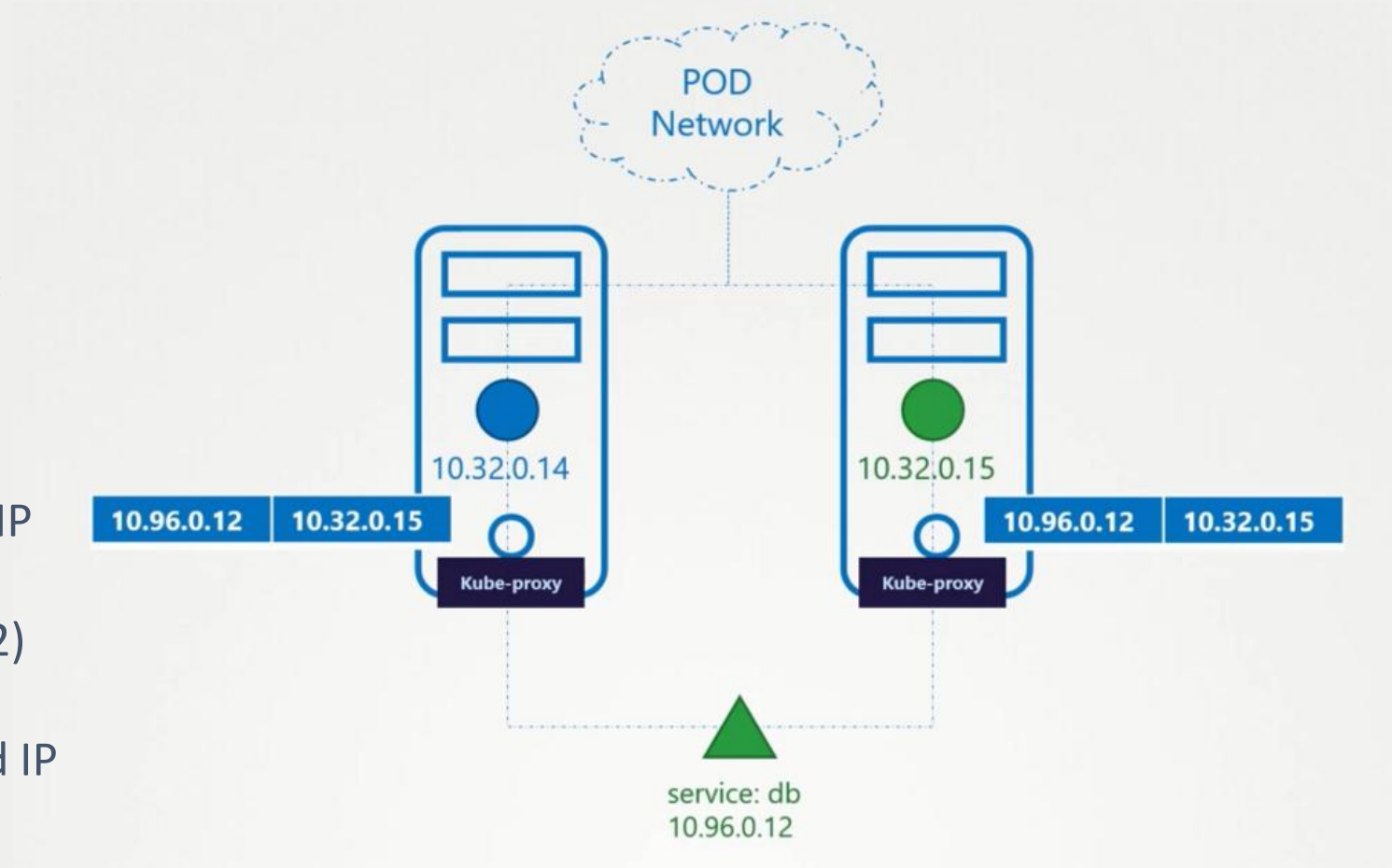
1. First, we create a Service.
2. Then, Service gets a ClusterIP.



1. First, we create a Service.
2. Then, Service gets a ClusterIP.



1. First, we create a Service.
2. Then, Service gets a ClusterIP.
3. Kube-proxy sets up rules on every node:
  - Creates iptables/IPVS rules
  - Routes traffic from Service IP to desired IP
4. Pod calls the stable Service IP (10.96.0.12)
5. kube-proxy translates it to the actual pod IP (10.32.0.15)
6. kube-proxy routes traffic through the pod network to reach green pod.



PODs

# Assumptions

---



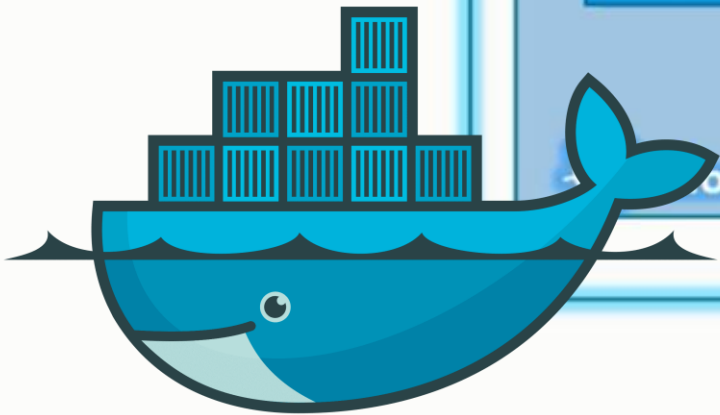
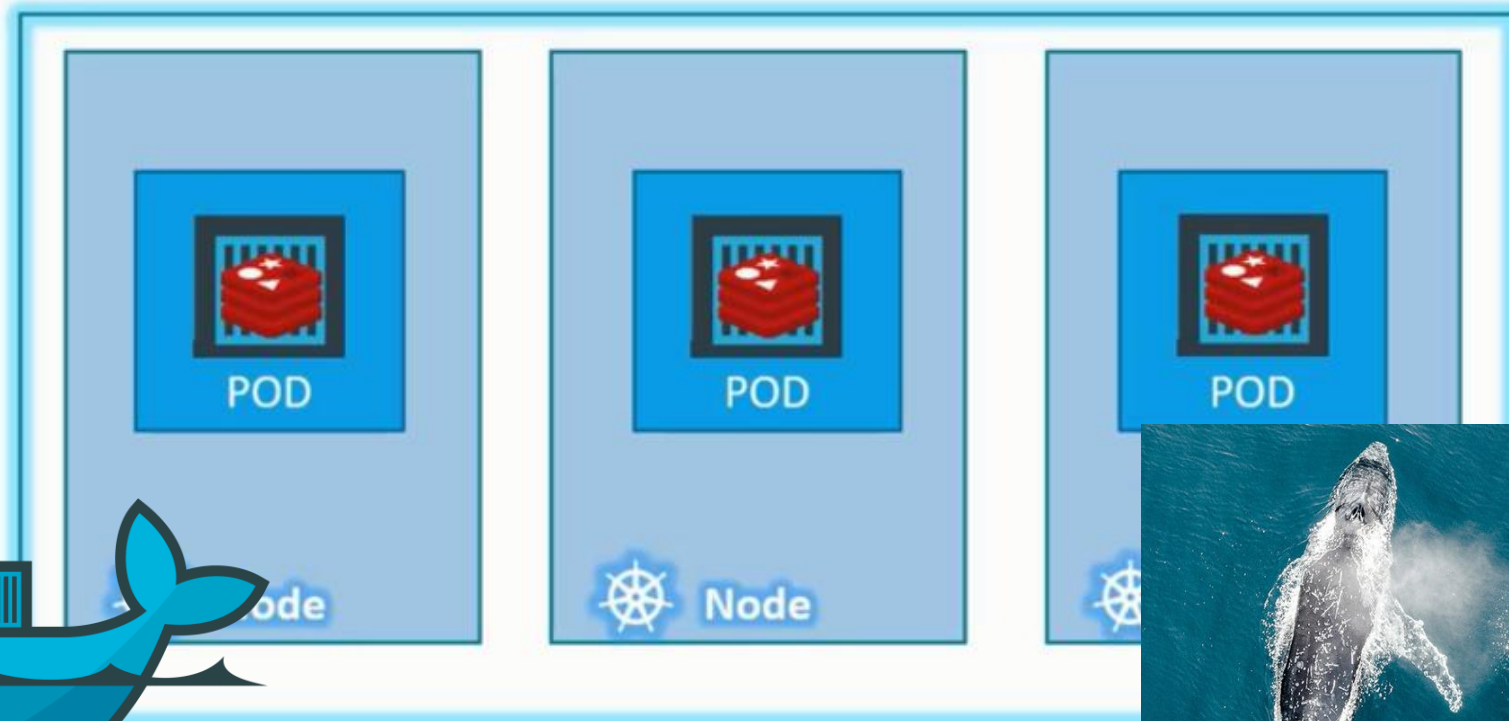
Docker Image



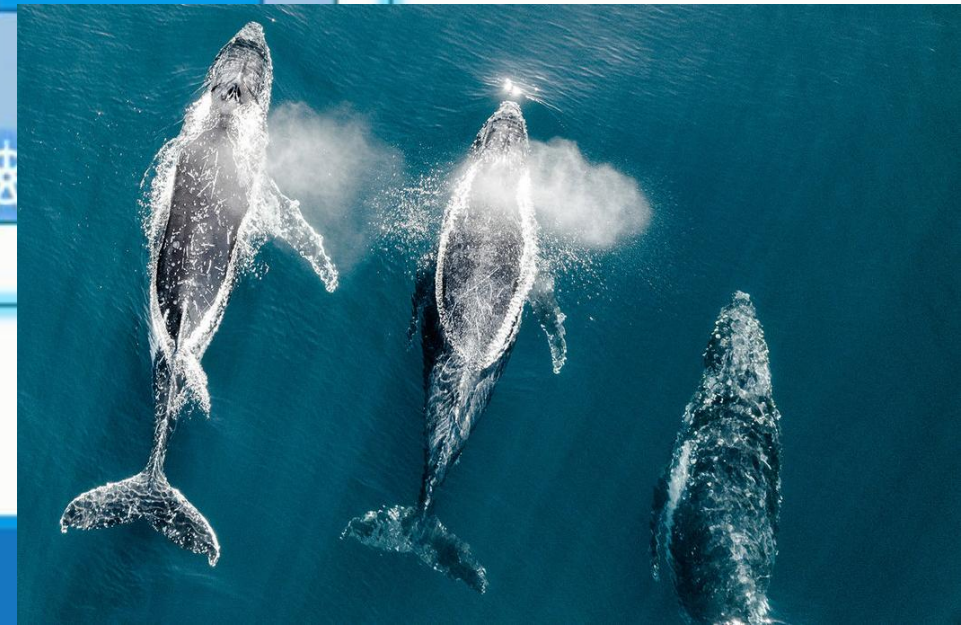
Kubernetes Cluster

# POD

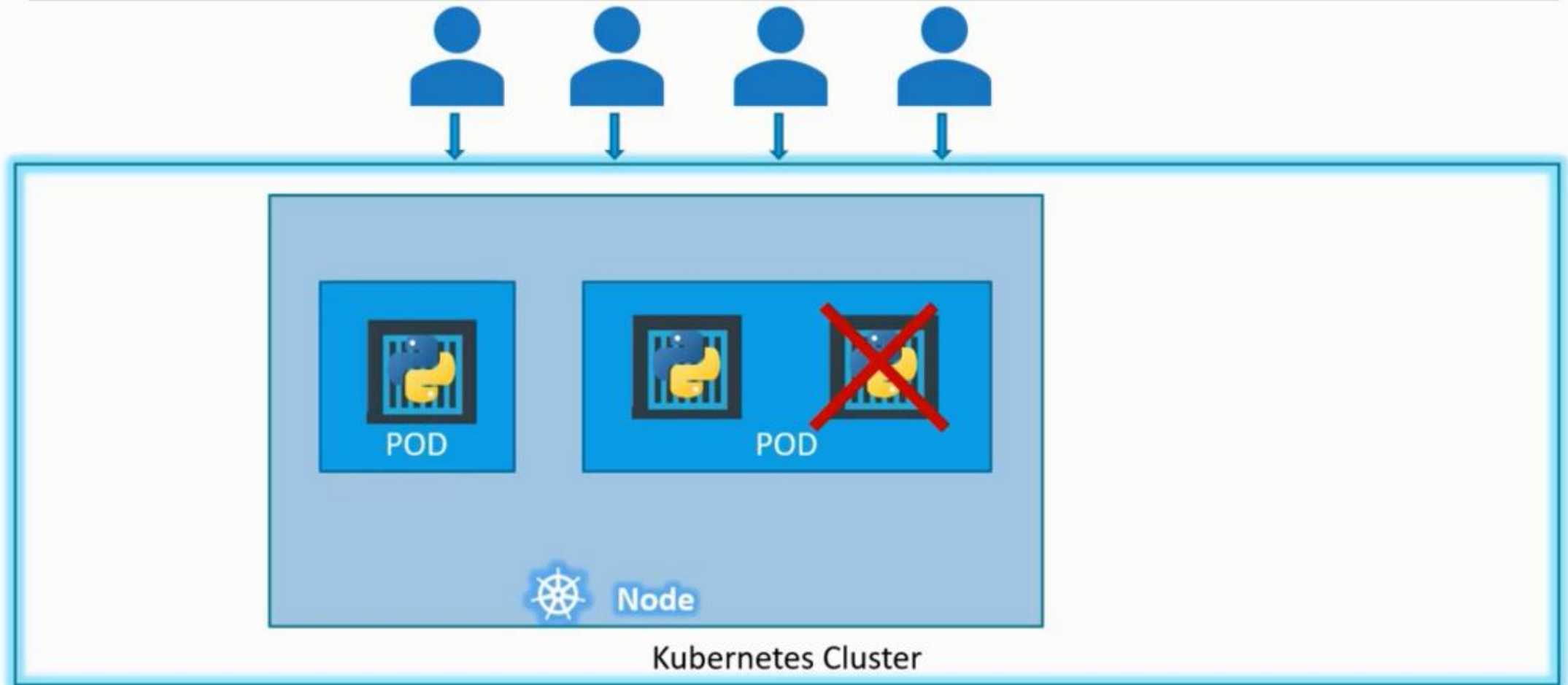
Pod: a group of usually related whales or dolphins of the same species that tend to travel together



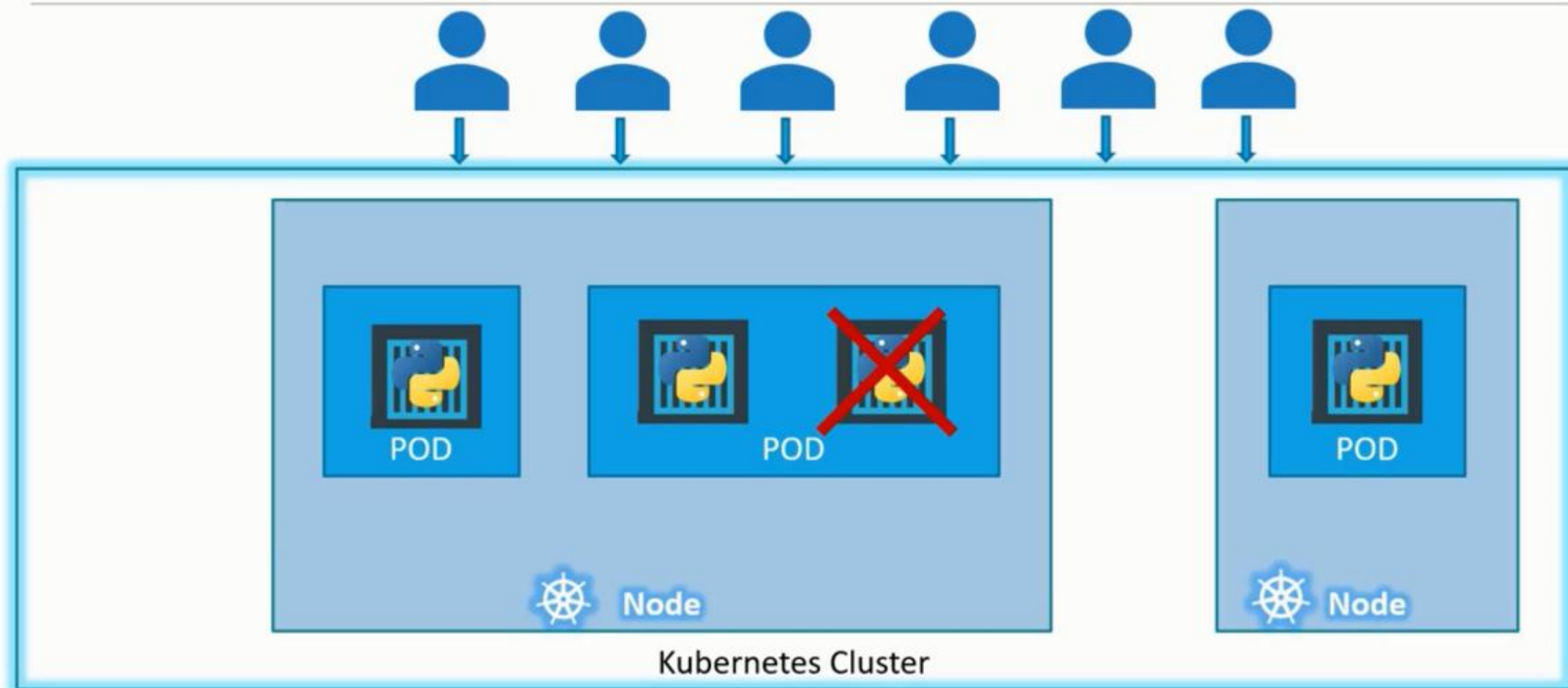
# docker



# POD



# POD



# PODs Again!

```
docker run python-app
```

```
docker run python-app
```

```
docker run python-app
```

```
docker run python-app
```

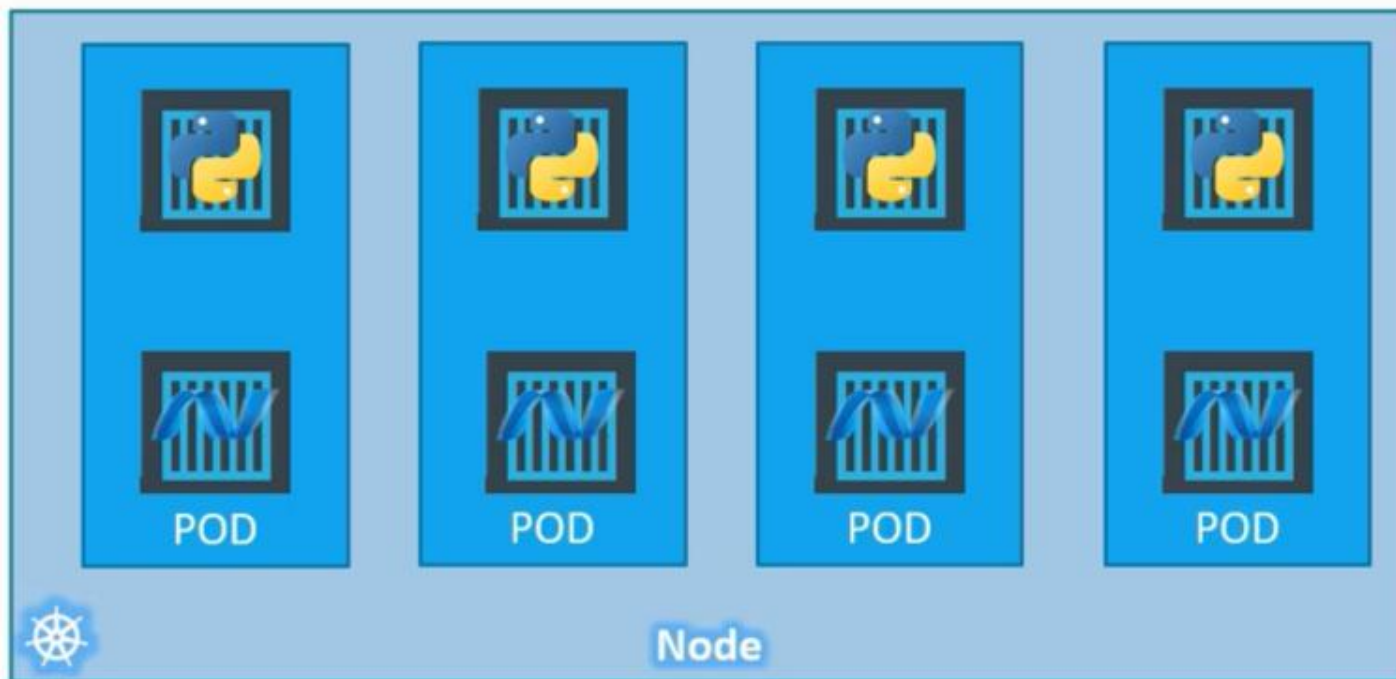
```
docker run helper -link app1
```

```
docker run helper -link app2
```

```
docker run helper -link app3
```

```
docker run helper -link app4
```

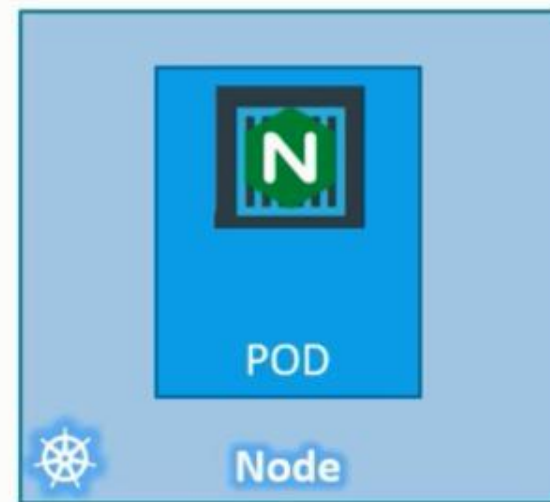
| App     | Helper | Volume |
|---------|--------|--------|
| Python1 | App1   | Vol1   |
| Python2 | App2   | Vol2   |



# kubectl

---

```
kubectl run nginx --image nginx
```



# kubectl

```
kubectl run nginx --image nginx
```

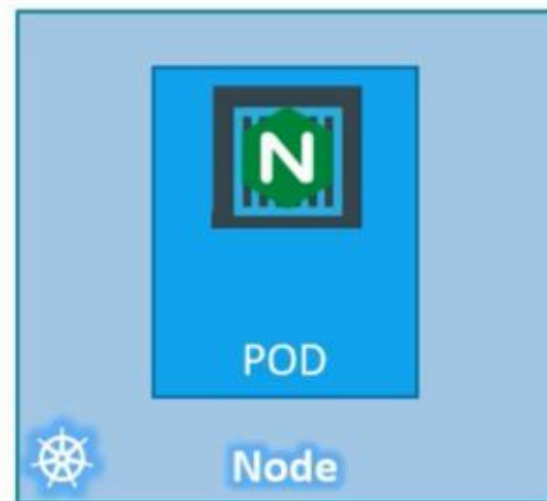
```
kubectl get pods
```

```
C:\Kubernetes>kubectl get pods
```

| NAME                 | READY | STATUS            | RESTARTS | AGE |
|----------------------|-------|-------------------|----------|-----|
| nginx-8586cf59-whssr | 0/1   | ContainerCreating | 0        | 3s  |

```
C:\Kubernetes>kubectl get pods
```

| NAME                 | READY | STATUS  | RESTARTS | AGE |
|----------------------|-------|---------|----------|-----|
| nginx-8586cf59-whssr | 1/1   | Running | 0        | 8s  |



# YAML in Kubernetes

---

pod-definition.yml

```
apiVersion:
```

```
kind:
```

```
metadata:
```

```
spec:
```

# YAML in Kubernetes

---

```
pod-definition.yml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
```

String

String

Dictionary

| Kind       | Version |
|------------|---------|
| POD        | v1      |
| Service    | v1      |
| ReplicaSet | apps/v1 |
| Deployment | apps/v1 |
|            |         |

# YAML in Kubernetes

---

pod-definition.yml

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
```



| Kind       | Version |
|------------|---------|
| POD        | v1      |
| Service    | v1      |
| ReplicaSet | apps/v1 |
| Deployment | apps/v1 |
|            |         |

# YAML in Kubernetes

---

pod-definition.yml

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
```



| Kind       | Version |
|------------|---------|
| POD        | v1      |
| Service    | v1      |
| ReplicaSet | apps/v1 |
| Deployment | apps/v1 |
|            |         |

# YAML in Kubernetes

---

pod-definition.yml

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers: ————— List/Array
  - name: nginx-container
    image: nginx
```

| Kind       | Version |
|------------|---------|
| POD        | v1      |
| Service    | v1      |
| ReplicaSet | apps/v1 |
| Deployment | apps/v1 |
|            |         |

# YAML in Kubernetes

---

pod-definition.yml

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

1<sup>st</sup> Item in List

| Kind       | Version |
|------------|---------|
| POD        | v1      |
| Service    | v1      |
| ReplicaSet | apps/v1 |
| Deployment | apps/v1 |
|            |         |

pod-definition.yml

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
    - name: nginx-container
      image: nginx
```

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
    - name: nginx-container
      image: nginx
    - name: redis-container
      image: redis
    - name: busybox-container
      image: busybox
      command: ['sh', '-c', 'echo Hello Kubernetes! && sleep 3600']
```



# YAML in Kubernetes

---

pod-definition.yml

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

```
kubectl create -f pod-definition.yml
```

| Kind       | Version |
|------------|---------|
| POD        | v1      |
| Service    | v1      |
| ReplicaSet | apps/v1 |
| Deployment | apps/v1 |
|            |         |

# Commands

```
> kubectl get pods
```

| NAME      | READY | STATUS  | RESTARTS | AGE |
|-----------|-------|---------|----------|-----|
| myapp-pod | 1/1   | Running | 0        | 20s |

```
> kubectl describe pod myapp-pod
```

```
Name:          myapp-pod
Namespace:     default
Node:          minikube/192.168.99.100
Start Time:    Sat, 03 Mar 2018 14:26:14 +0800
Labels:        app=myapp
               name=myapp-pod
Annotations:   <none>
Status:        Running
IP:            172.17.0.24
Containers:
  nginx:
    Container ID:  docker://830bb56c8c42a86b4bb70e9c1488fae1bc38663e4918b6c2f5a783e7688b8c9d
    Image:         nginx
    Image ID:      docker-pullable://nginx@sha256:4771d09578c7c6a65299e110b3ee1c0a2592f5ea2618d23e4ffe7a4cab1ce5de
    Port:          <none>
    State:         Running
      Started:     Sat, 03 Mar 2018 14:26:21 +0800
    Ready:         True
    Restart Count: 0
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from default-token-x95w7 (ro)
Conditions:
  Type             Status
  Initialized       True
  Ready            True
  PodScheduled     True
Events:
  Type    Reason            Age   From                  Message
  ----    -
  Normal  Scheduled         34s   default-scheduler    Successfully assigned myapp-pod to minikube
  Normal  SuccessfulMountVolume 33s   kubelet, minikube    MountVolume.SetUp succeeded for volume "default-token-x95w7"
  Normal  Pulling           33s   kubelet, minikube    pulling image "nginx"
  Normal  Pulled            27s   kubelet, minikube    Successfully pulled image "nginx"
  Normal  Created           27s   kubelet, minikube    Created container
  Normal  Started           27s   kubelet, minikube    Started container
```

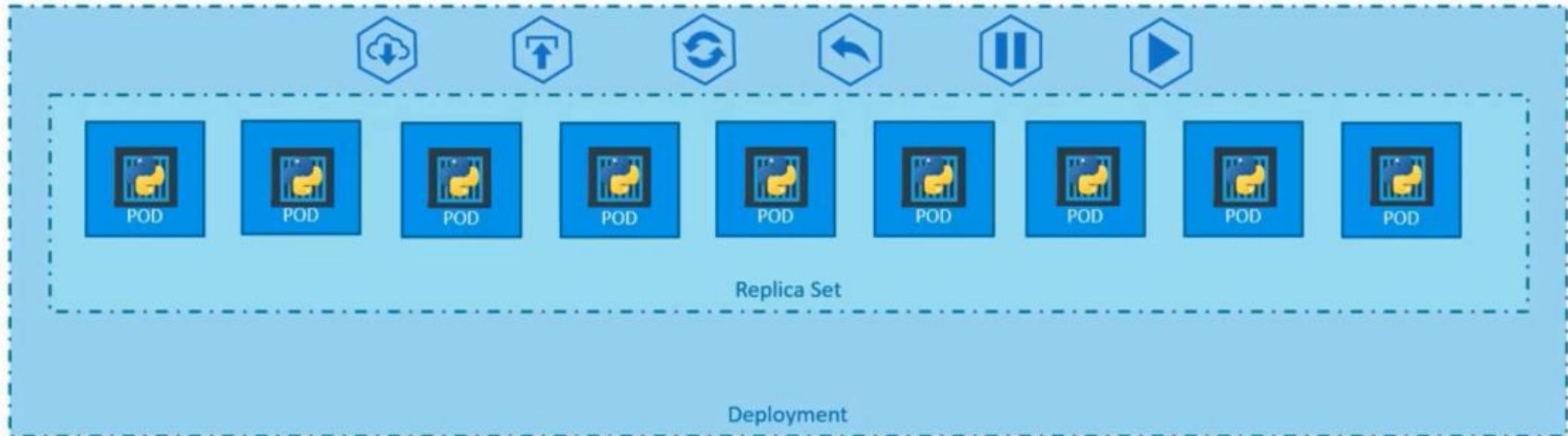
# Deployment

# Deployment



1. **Create** - Adds new pods to the deployment
2. **Scale Up** - Increases the number of pod replicas
3. **Update/Rollout** - Deploys a new version of the application
4. **Rollback** - Reverts to a previous version
5. **Pause** - Temporarily stops the deployment process
6. **Resume/Play** - Continues or restarts the deployment

# Deployment



Deployments manage Replica-Sets and Replica-Sets manage Pods

# Definition

---

deployment-definition.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
        - name: nginx-container
          image: nginx
  replicas: 3
  selector:
    matchLabels:
      type: front-end
```

# Definition

```
> kubectl create -f deployment-definition.yml
```

```
deployment "myapp-deployment" created
```

```
> kubectl get deployments
```

| NAME             | DESIRED | CURRENT | UP-TO-DATE | AVAILABLE | AGE |
|------------------|---------|---------|------------|-----------|-----|
| myapp-deployment | 3       | 3       | 3          | 3         | 21s |

```
deployment-definition.yml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: myapp-deployment
```

```
  labels:
```

```
    app: myapp
```

```
    type: front-end
```

```
spec:
```

```
  template:
```

```
    metadata:
```

```
      name: myapp-pod
```

```
      labels:
```

```
        app: myapp
```

```
        type: front-end
```

```
    spec:
```

```
      containers:
```

```
        - name: nginx-container
```

```
          image: nginx
```

```
replicas: 3
```

```
selector:
```

```
  matchLabels:
```

```
    type: front-end
```

# Definition

```
> kubectl create -f deployment-definition.yml
```

```
deployment "myapp-deployment" created
```

```
> kubectl get deployments
```

| NAME             | DESIRED | CURRENT | UP-TO-DATE | AVAILABLE | AGE |
|------------------|---------|---------|------------|-----------|-----|
| myapp-deployment | 3       | 3       | 3          | 3         | 21s |

```
> kubectl get replicaset
```

| NAME                        | DESIRED | CURRENT | READY | AGE |
|-----------------------------|---------|---------|-------|-----|
| myapp-deployment-6795844b58 | 3       | 3       | 3     | 2m  |

```
deployment-definition.yml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: myapp-deployment
```

```
  labels:
```

```
    app: myapp
```

```
    type: front-end
```

```
spec:
```

```
  template:
```

```
    metadata:
```

```
      name: myapp-pod
```

```
      labels:
```

```
        app: myapp
```

```
        type: front-end
```

```
    spec:
```

```
      containers:
```

```
        - name: nginx-container
```

```
          image: nginx
```

```
replicas: 3
```

```
selector:
```

```
  matchLabels:
```

```
    type: front-end
```

# Definition

```
> kubectl create -f deployment-definition.yml
```

```
deployment "myapp-deployment" created
```

```
> kubectl get deployments
```

| NAME             | DESIRED | CURRENT | UP-TO-DATE | AVAILABLE | AGE |
|------------------|---------|---------|------------|-----------|-----|
| myapp-deployment | 3       | 3       | 3          | 3         | 21s |

```
> kubectl get replicaset
```

| NAME                        | DESIRED | CURRENT | READY | AGE |
|-----------------------------|---------|---------|-------|-----|
| myapp-deployment-6795844b58 | 3       | 3       | 3     | 2m  |

```
> kubectl get pods
```

| NAME                               | READY | STATUS  | RESTARTS | AGE |
|------------------------------------|-------|---------|----------|-----|
| myapp-deployment-6795844b58-5rbj1  | 1/1   | Running | 0        | 2m  |
| myapp-deployment-6795844b58-h4w55  | 1/1   | Running | 0        | 2m  |
| myapp-deployment-6795844b58-1fj hv | 1/1   | Running | 0        | 2m  |

```
deployment-definition.yml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: myapp-deployment
```

```
  labels:
```

```
    app: myapp
```

```
    type: front-end
```

```
spec:
```

```
  template:
```

```
    metadata:
```

```
      name: myapp-pod
```

```
      labels:
```

```
        app: myapp
```

```
        type: front-end
```

```
    spec:
```

```
      containers:
```

```
        - name: nginx-container
```

```
          image: nginx
```

```
replicas: 3
```

```
selector:
```

```
  matchLabels:
```

```
    type: front-end
```

# commands

---

```
> kubectl get all
```

| NAME                    | DESIRED | CURRENT | UP-TO-DATE | AVAILABLE | AGE |
|-------------------------|---------|---------|------------|-----------|-----|
| deploy/myapp-deployment | 3       | 3       | 3          | 3         | 9h  |

| NAME                           | DESIRED | CURRENT | READY | AGE |
|--------------------------------|---------|---------|-------|-----|
| rs/myapp-deployment-6795844b58 | 3       | 3       | 3     | 9h  |

| NAME                                  | READY | STATUS  | RESTARTS | AGE |
|---------------------------------------|-------|---------|----------|-----|
| po/myapp-deployment-6795844b58-5rbj1  | 1/1   | Running | 0        | 9h  |
| po/myapp-deployment-6795844b58-h4w55  | 1/1   | Running | 0        | 9h  |
| po/myapp-deployment-6795844b58-lfj hv | 1/1   | Running | 0        | 9h  |

# Namespaces

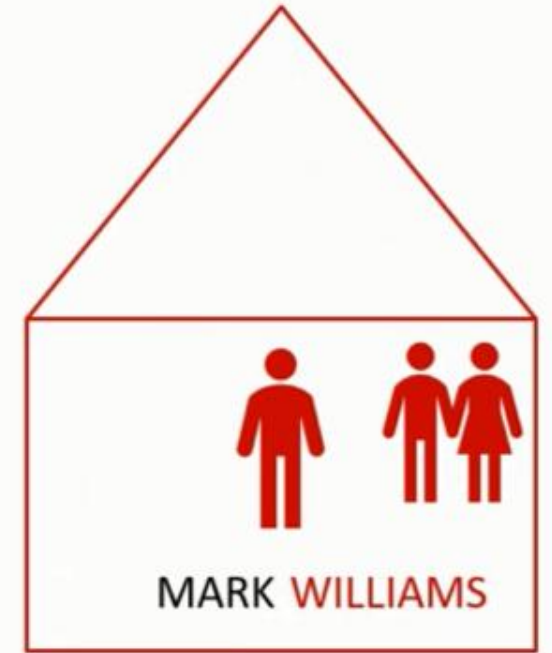
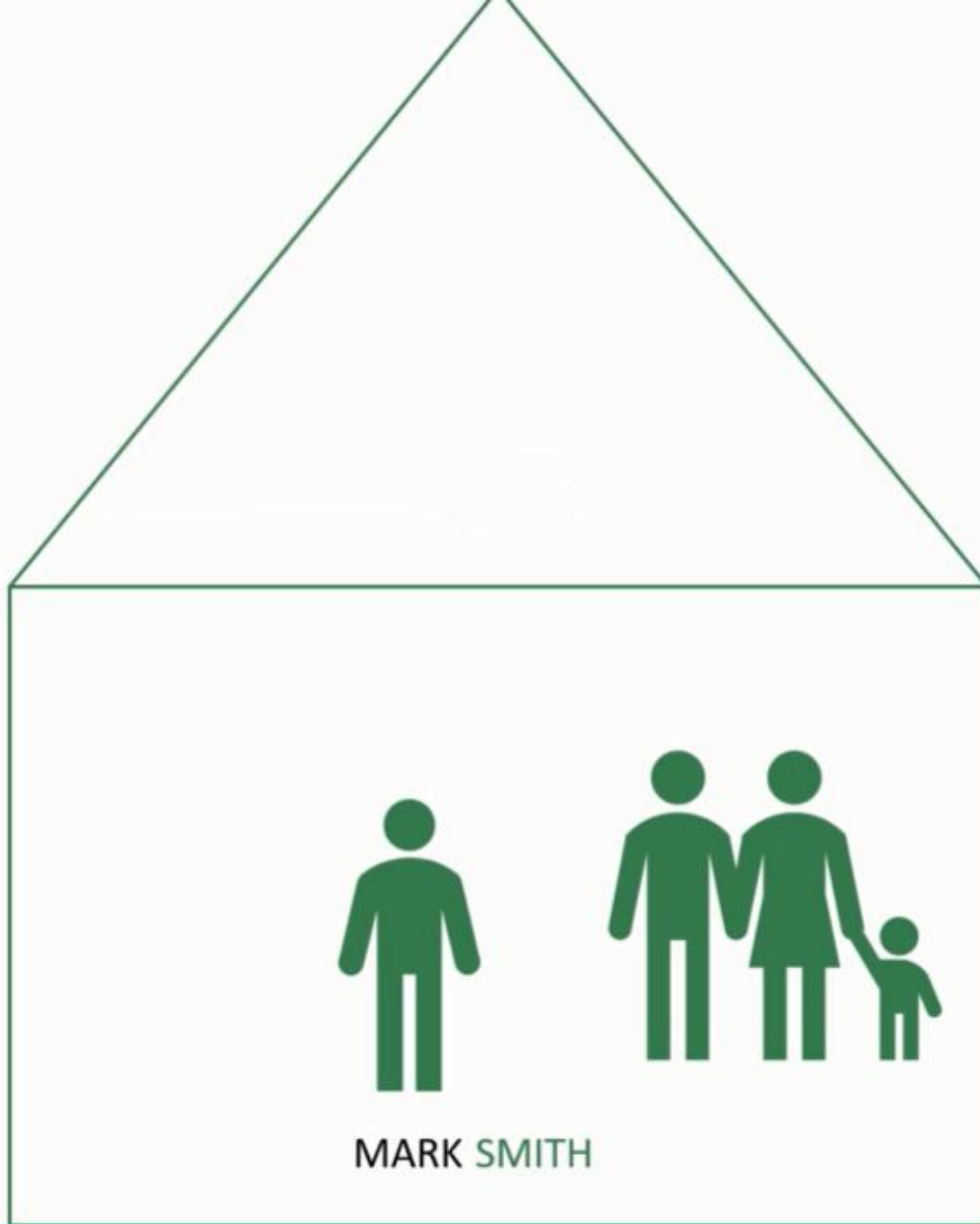


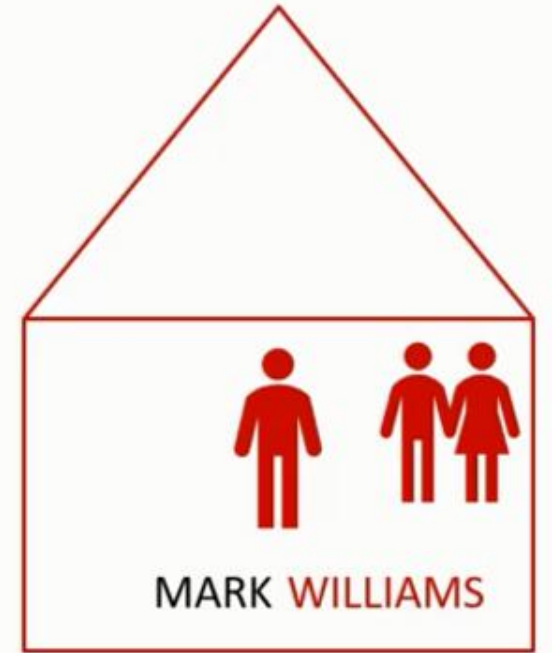
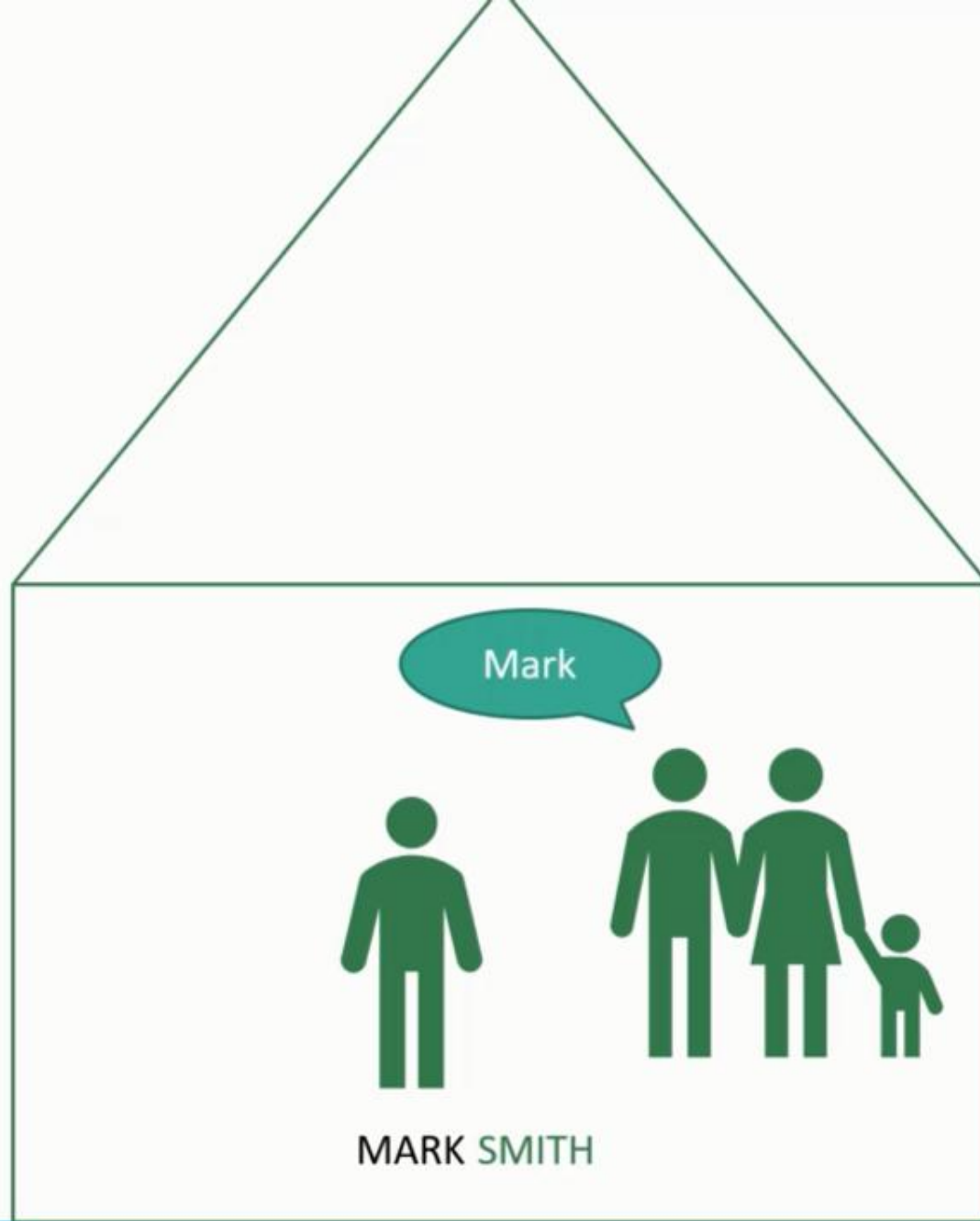
MARK SMITH

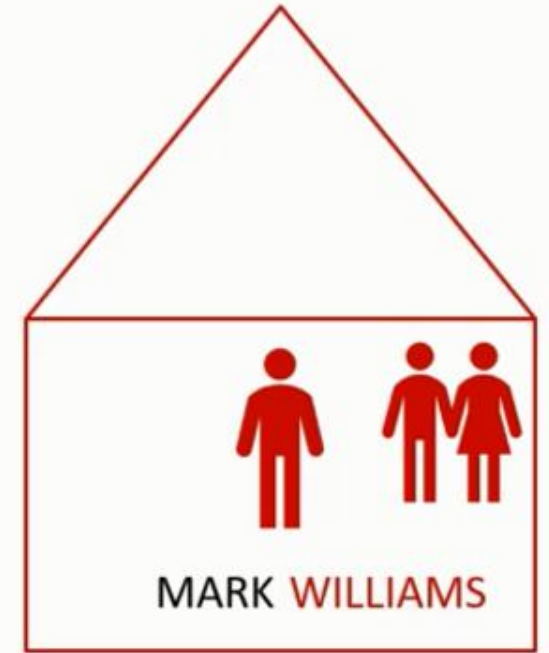
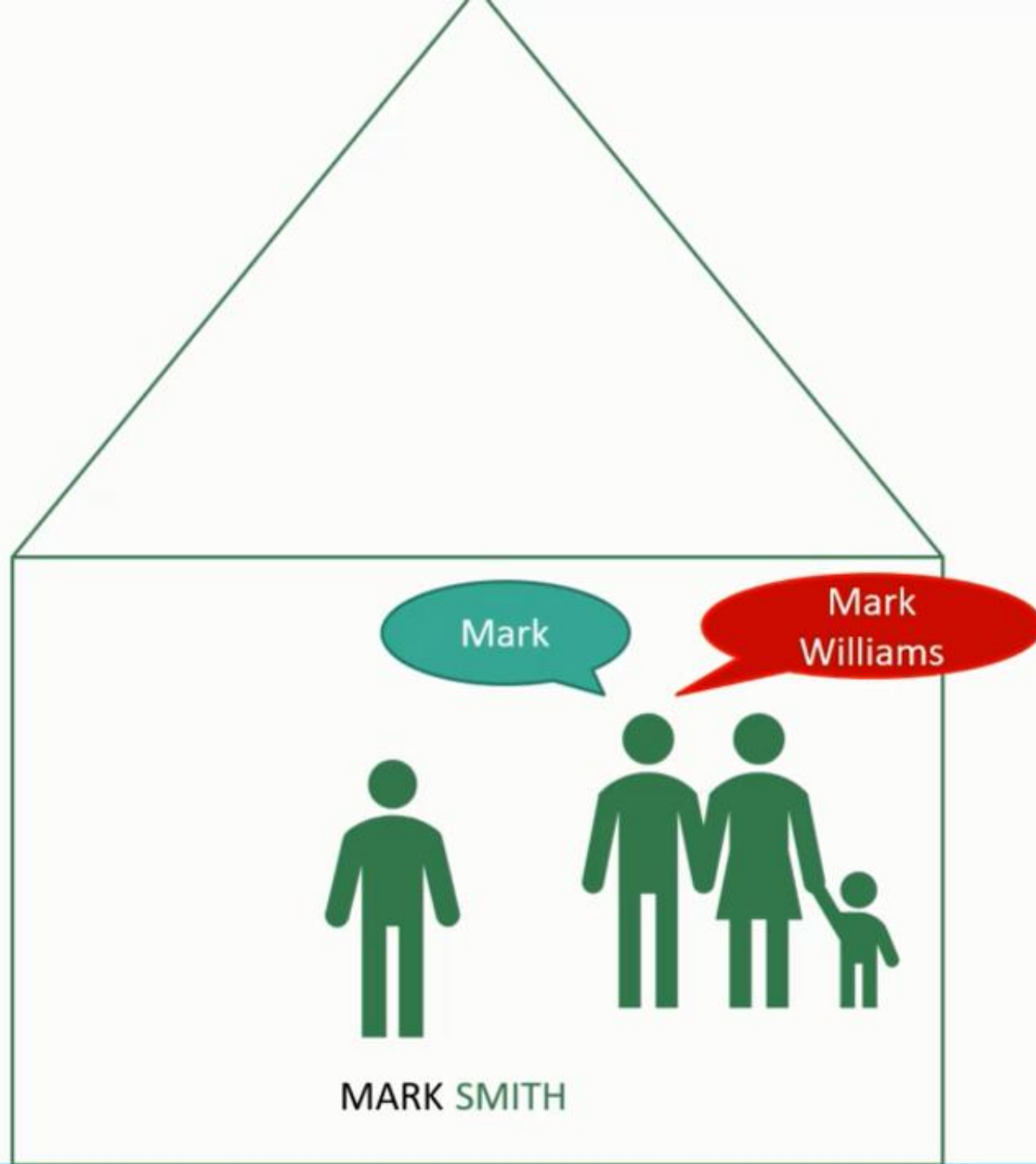


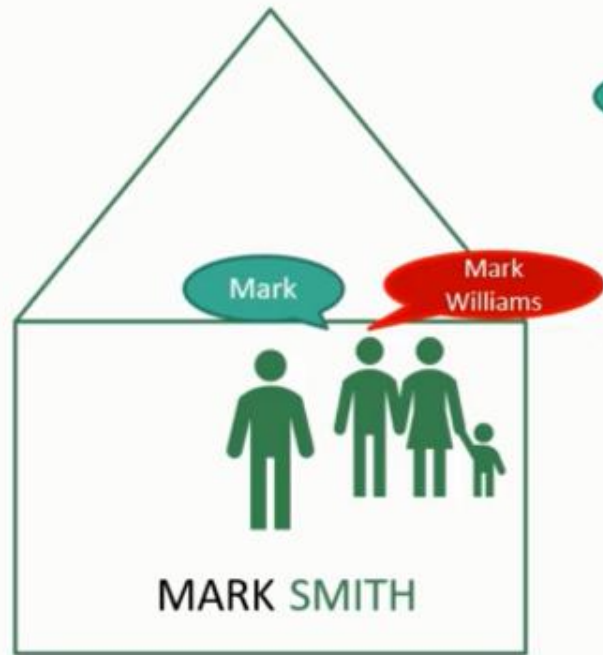
MARK WILLIAMS

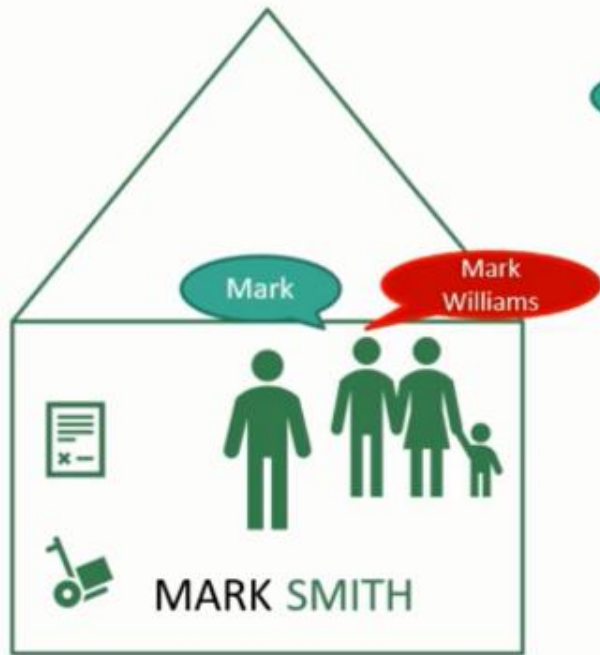


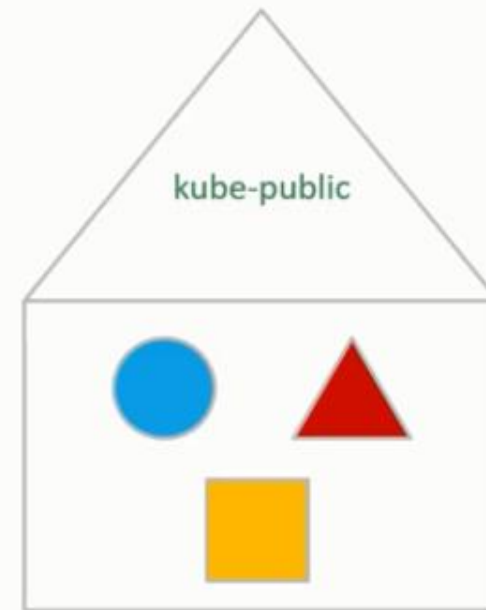
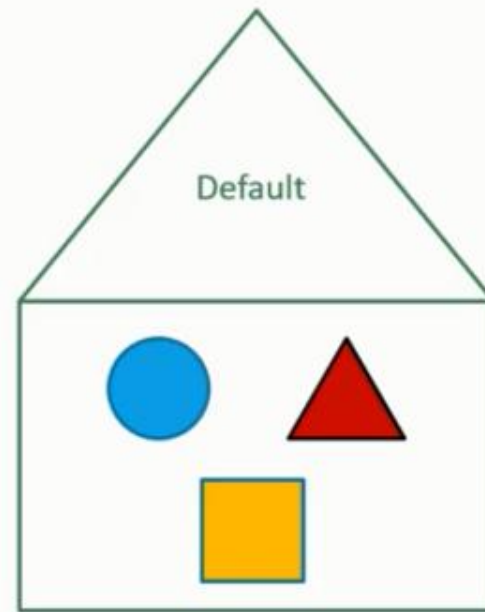
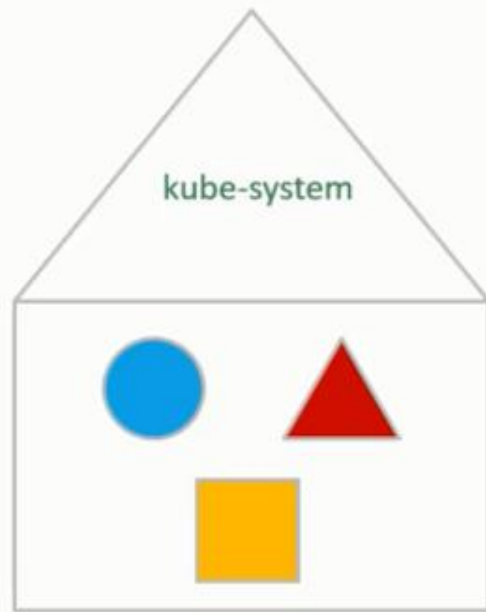




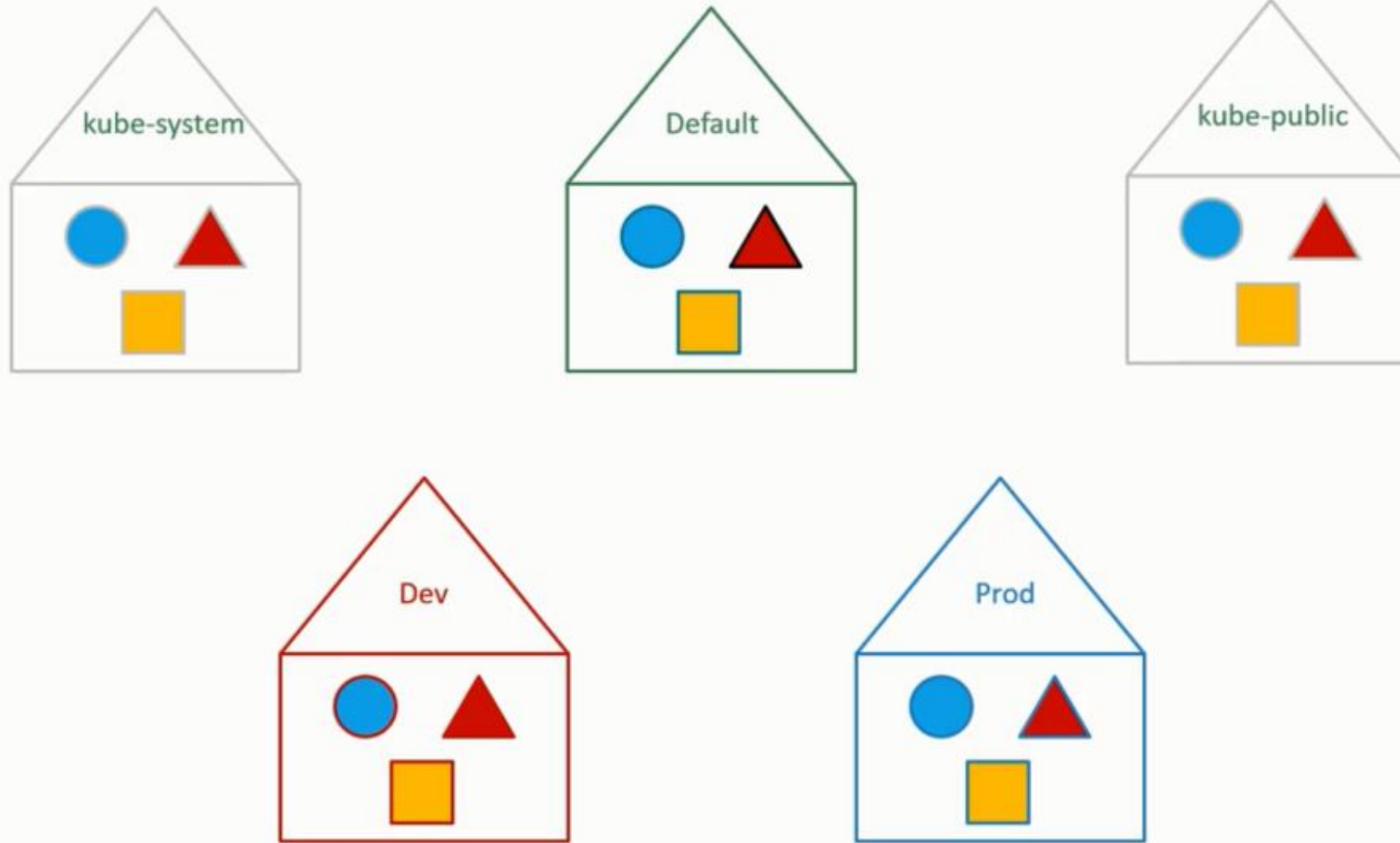




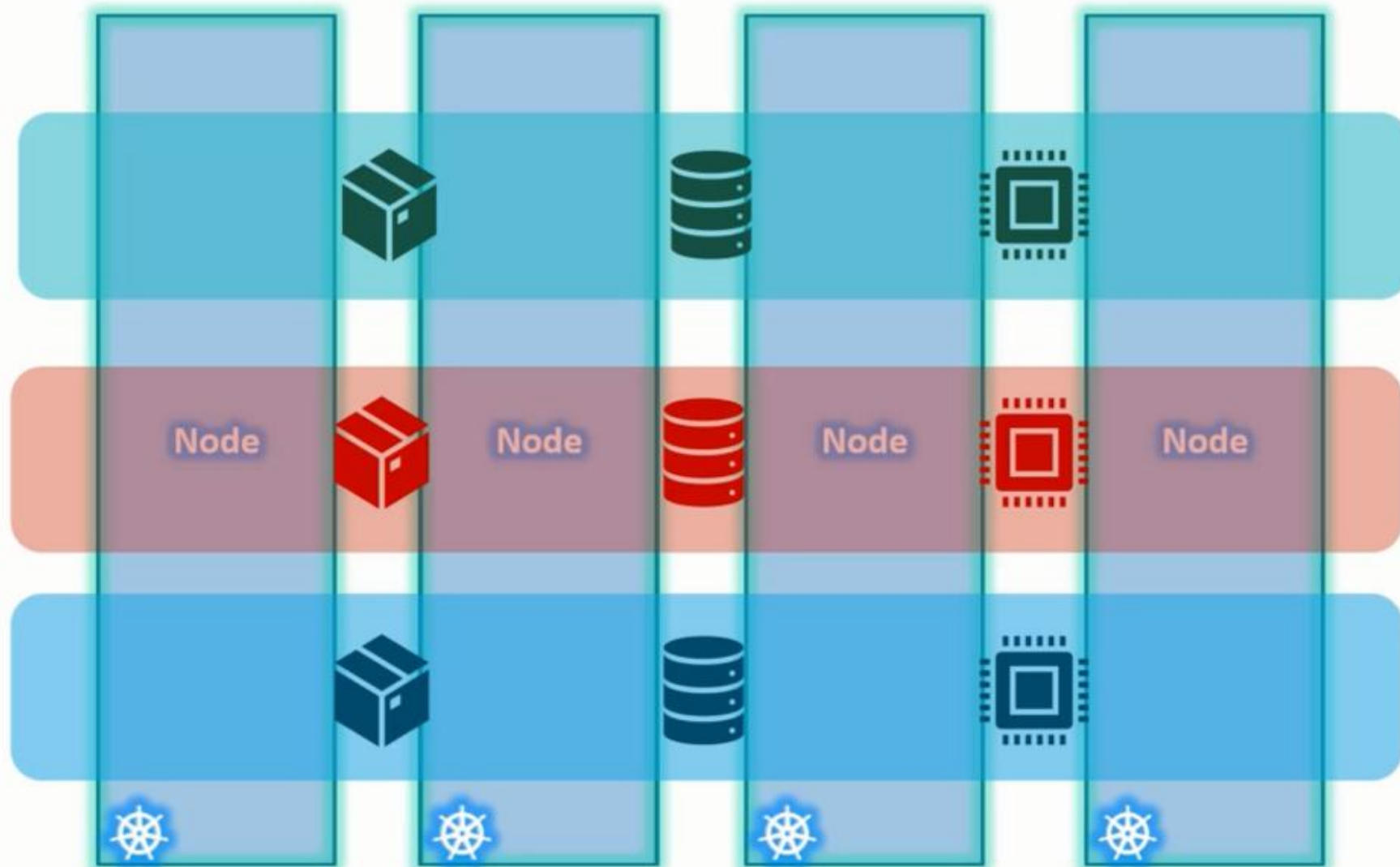
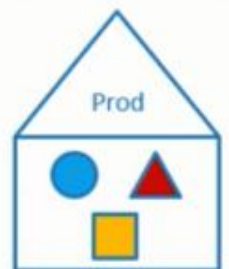
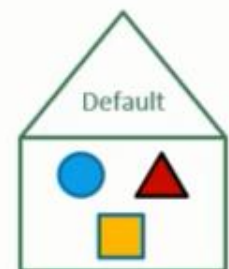




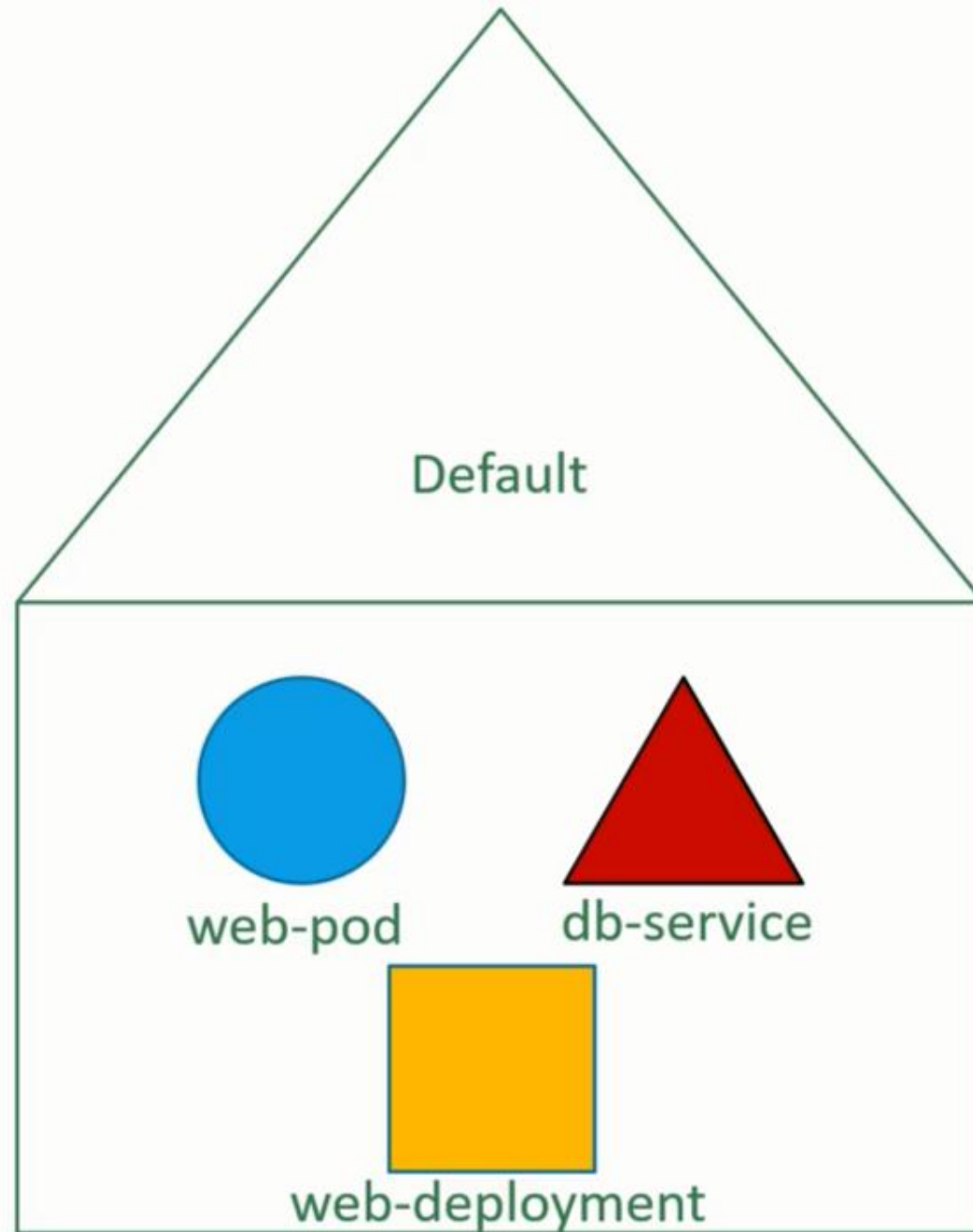
# Namespace - Isolation



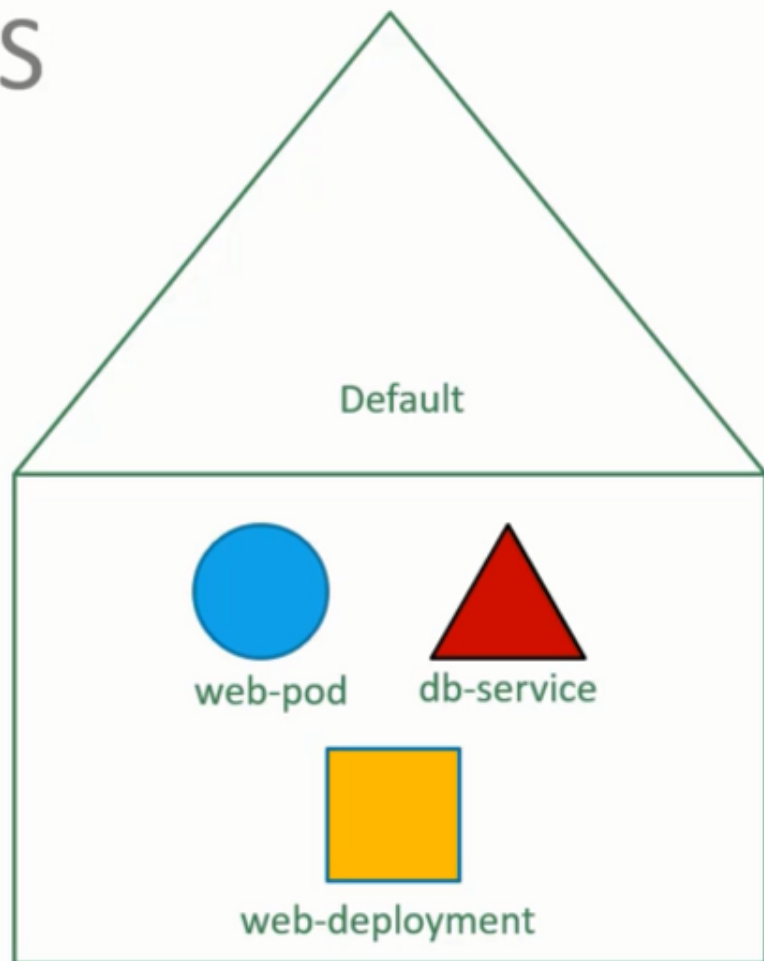
# Namespace – Resource Limits



# DNS

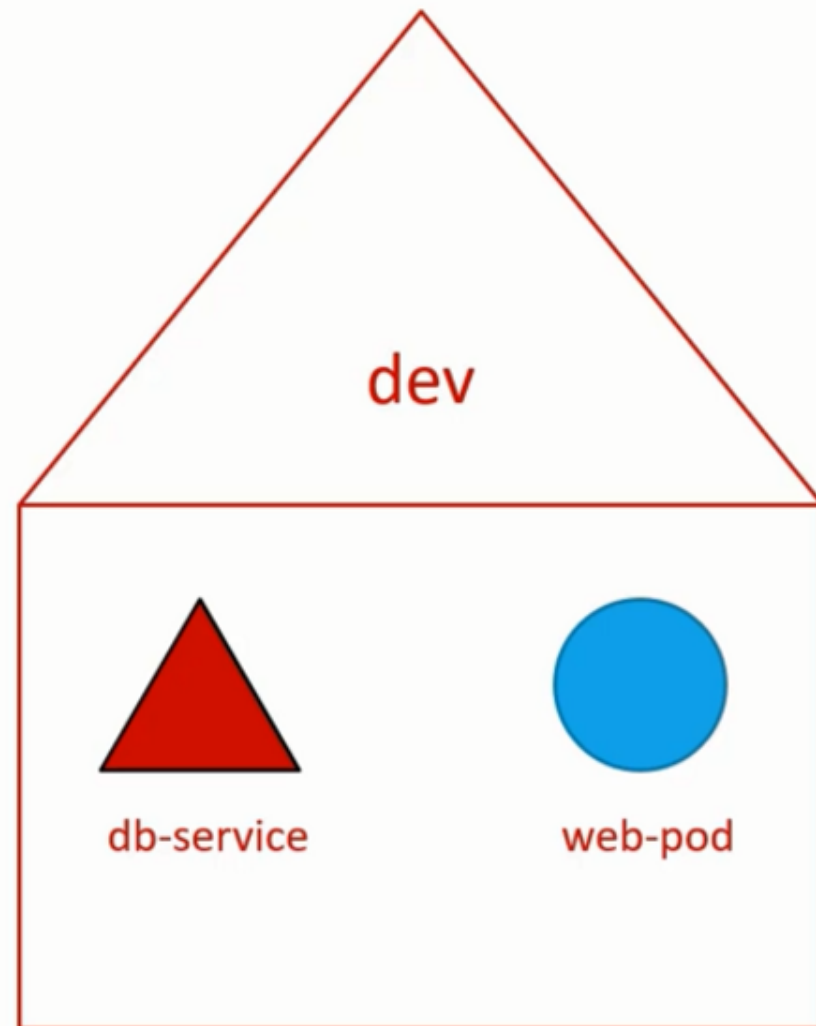


# DNS



```
mysql.connect("db-service")
```

```
mysql.connect("db-service.dev.svc.cluster.local")
```



# DNS

```
mysql.connect("db-service.dev.svc.cluster.local")
```

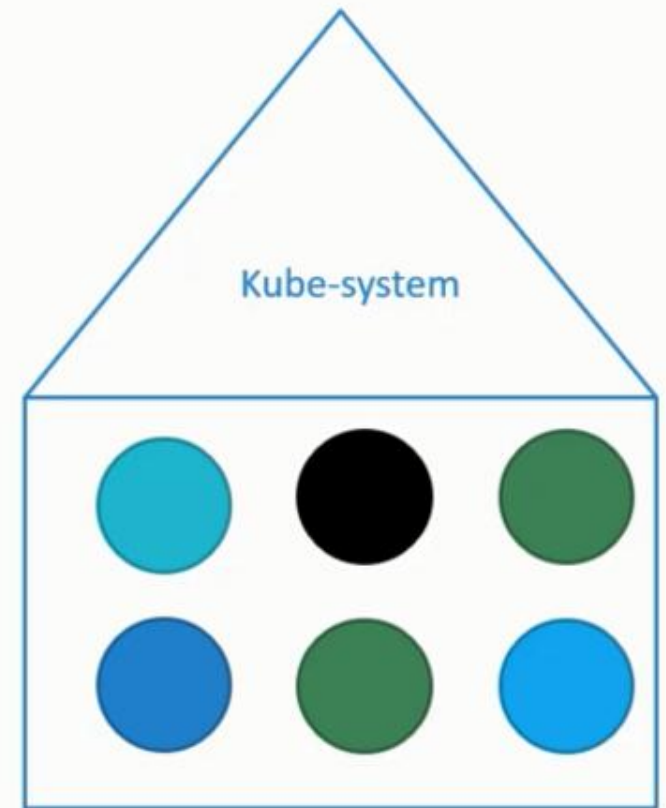
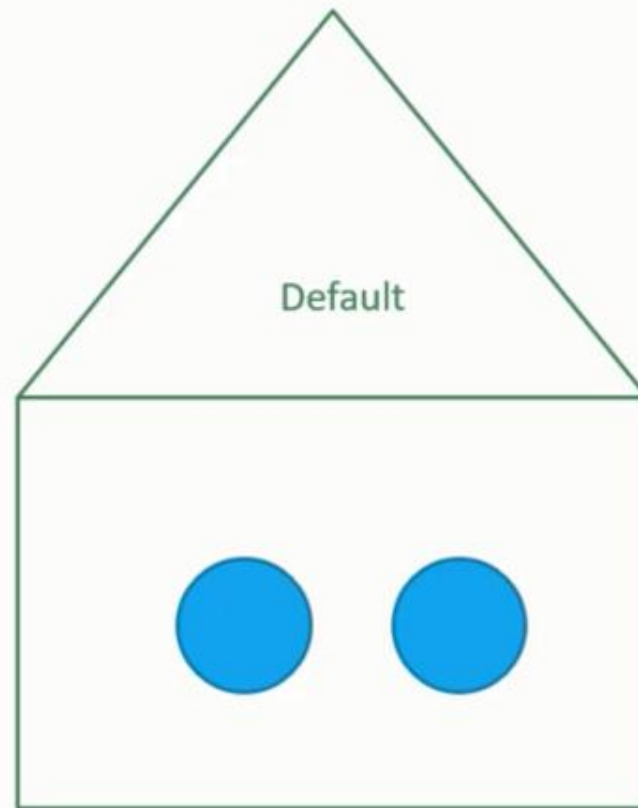


```
> kubectl get pods
```

| NAME  | READY | STATUS  | RESTARTS | AGE |
|-------|-------|---------|----------|-----|
| Pod-1 | 1/1   | Running | 0        | 3d  |
| Pod-2 | 1/1   | Running | 0        | 3d  |

```
> kubectl get pods --namespace=kube-system
```

| NAME                           | READY | STATUS  | RESTARTS |
|--------------------------------|-------|---------|----------|
| coredns-78fcd6894-92d52        | 1/1   | Running | 7        |
| coredns-78fcd6894-jx25g        | 1/1   | Running | 7        |
| etcd-master                    | 1/1   | Running | 7        |
| kube-apiserver-master          | 1/1   | Running | 7        |
| kube-controller-manager-master | 1/1   | Running | 7        |
| kube-flannel-ds-amd64-hz4cf    | 1/1   | Running | 14       |
| kube-proxy-4b8tn               | 1/1   | Running | 7        |
| kube-proxy-98db4               | 1/1   | Running | 7        |
| kube-proxy-jjrbs               | 1/1   | Running | 7        |
| kube-scheduler-master          | 1/1   | Running | 7        |



```
> kubectl create -f pod-definition.yml
```

```
pod/myapp-pod created
```

```
> kubectl create -f pod-definition.yml --namespace=dev
```

```
pod/myapp-pod created
```

```
pod-definition.yml
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: myapp-pod
```

```
  labels:
```

```
    app: myapp
```

```
    type: front-end
```

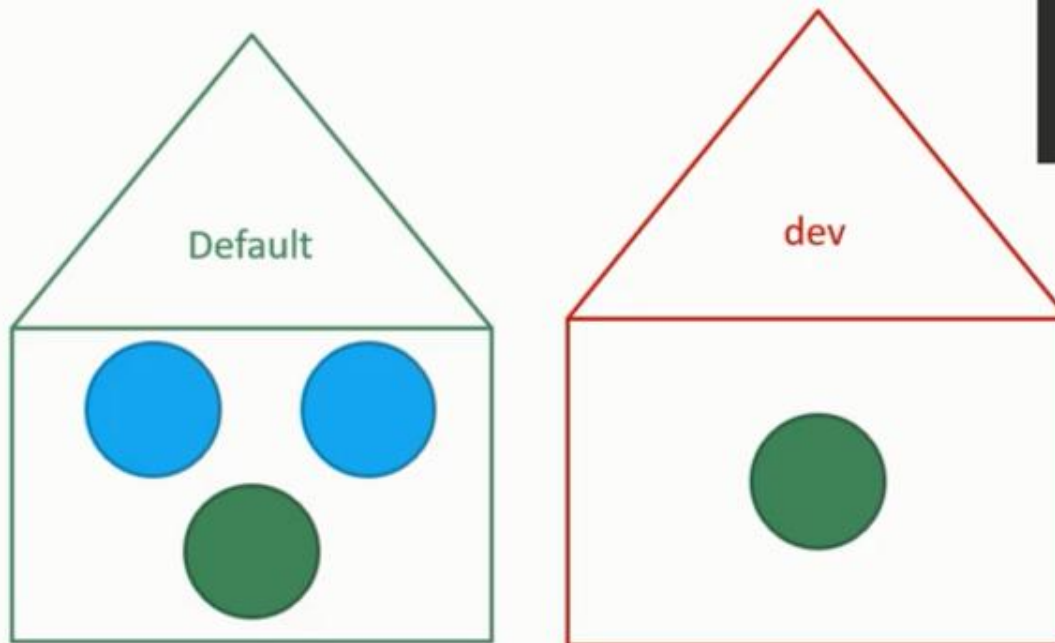
```
spec:
```

```
  containers:
```

```
  - name: nginx-container
```

```
    image: nginx
```

## Imperative



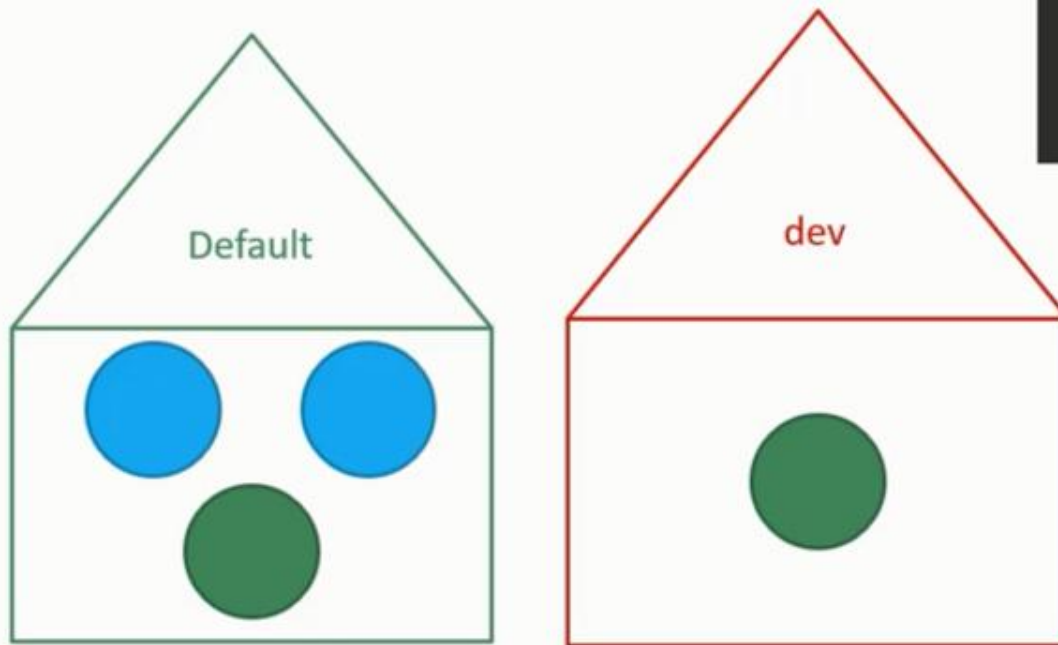
```
> kubectl create -f pod-definition.yml
```

```
pod/myapp-pod created
```

```
> kubectl create -f pod-definition.yml
```

```
pod/myapp-pod created
```

## Declarative



```
pod-definition.yml
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: myapp-pod
```

```
  namespace: dev
```

```
labels:
```

```
  app: myapp
```

```
  type: front-end
```

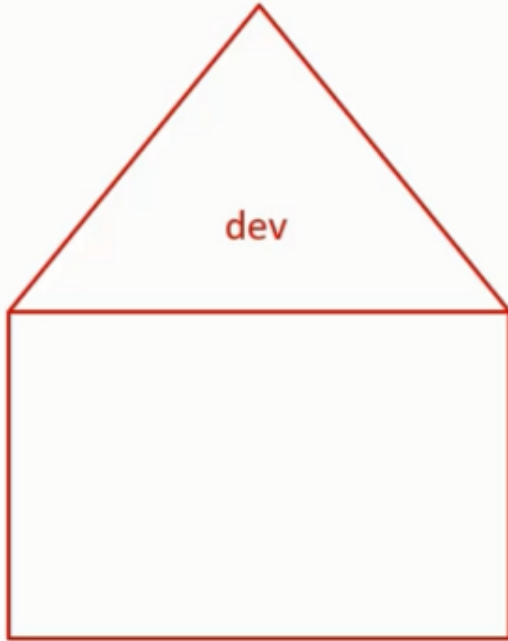
```
spec:
```

```
  containers:
```

```
  - name: nginx-container
```

```
    image: nginx
```

# Create Namespace



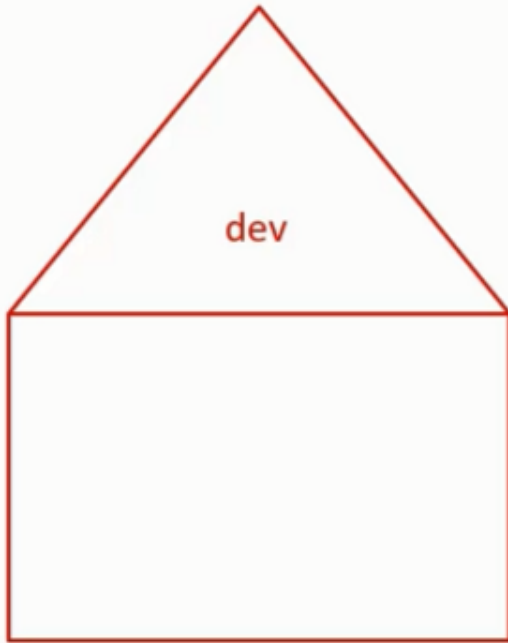
```
namespace-dev.yml
```

```
apiVersion: v1  
kind: Namespace  
metadata:  
  name: dev
```

```
> kubectl create -f namespace-dev.yml
```

```
namespace/dev created
```

# Create Namespace



```
namespace-dev.yml
```

```
apiVersion: v1  
kind: Namespace  
metadata:  
  name: dev
```

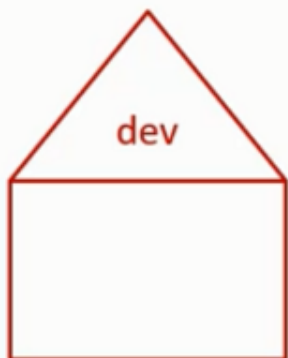
```
> kubectl create -f namespace-dev.yml
```

```
namespace/dev created
```

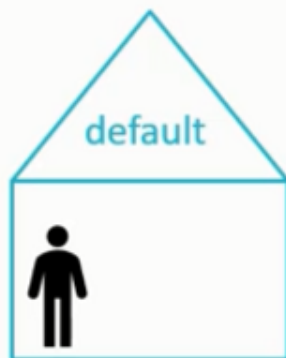
```
> kubectl create namespace dev
```

```
namespace/dev created
```

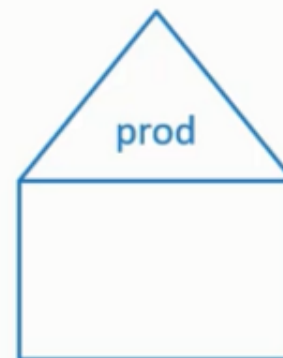
# Switch



```
> kubectl get pods --namespace=dev
```

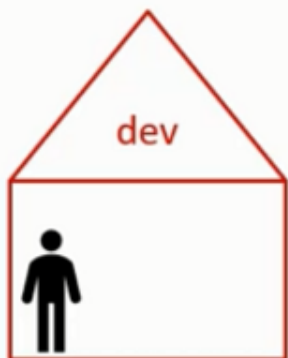


```
> kubectl get pods
```

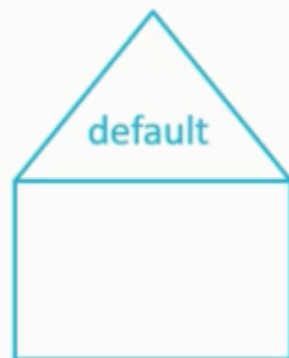


```
> kubectl get pods --namespace=prod
```

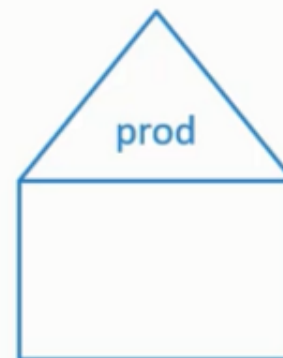
# Switch



```
> kubectl get pods --namespace=dev
```



```
> kubectl get pods
```

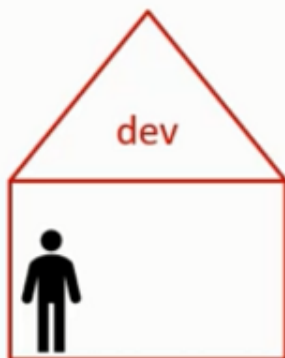


```
> kubectl get pods --namespace=prod
```

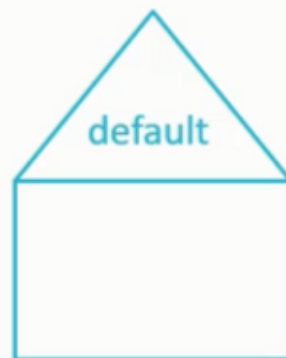
```
> kubectl config set-context $(kubectl config current-context) --namespace=dev
```

```
> kubectl get pod
```

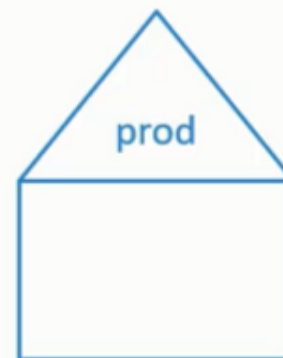
# Switch



```
> kubectl get pods --namespace=dev
```



```
> kubectl get pods
```



```
> kubectl get pods --namespace=prod
```

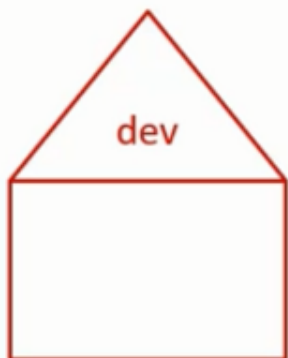
```
> kubectl config set-context $(kubectl config current-context) --namespace=dev
```

```
> kubectl get pods
```

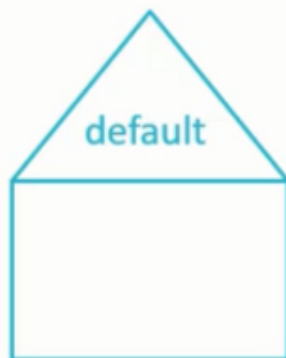
```
> kubectl get pods --namespace=default
```

```
> kubectl get pods --namespace=prod
```

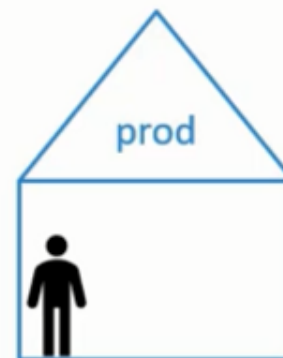
# Switch



```
> kubectl get pods --namespace=dev
```



```
> kubectl get pods
```



```
> kubectl get pods --namespace=prod
```

```
> kubectl config set-context $(kubectl config current-context) --namespace=dev
```

```
> kubectl get pods
```

```
> kubectl get pods --namespace=default
```

```
> kubectl get pods --namespace=prod
```

```
> kubectl config set-context $(kubectl config current-context) --namespace=prod
```

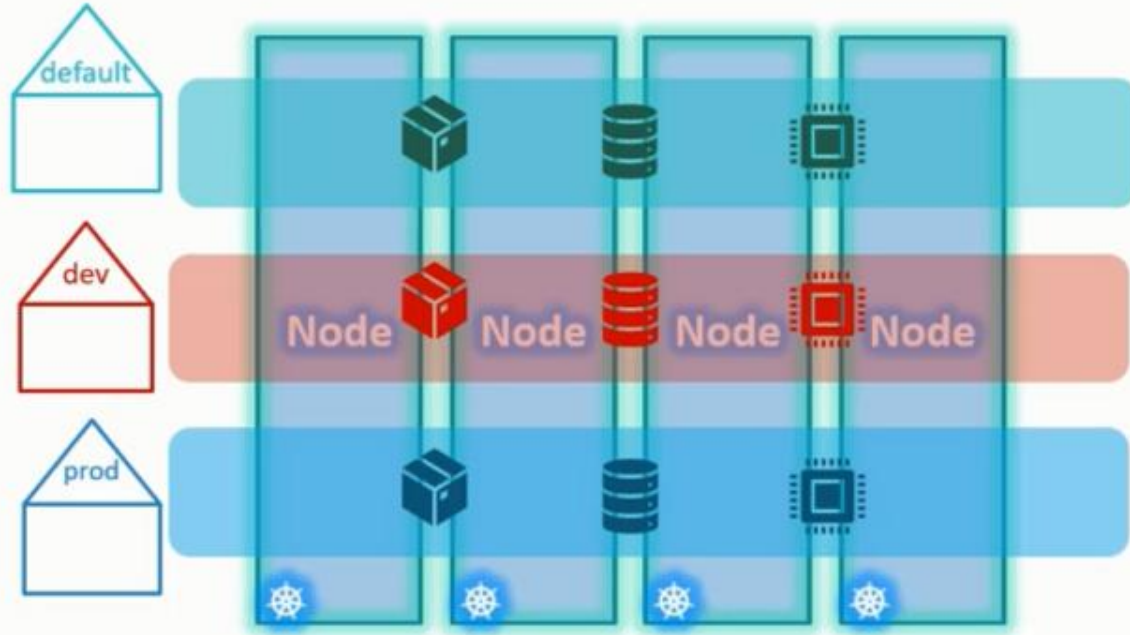
```
> kubectl get pods --namespace=dev
```

```
> kubectl get pods --namespace=default
```

```
> kubectl get pods
```

```
> kubectl get pods --all-namespaces
```

# Resource Quota



Compute-quota.yaml

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: dev
spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: 5Gi
    limits.cpu: "10"
    limits.memory: 10Gi
```

```
> kubectl create -f compute-quota.yaml
```

# Services

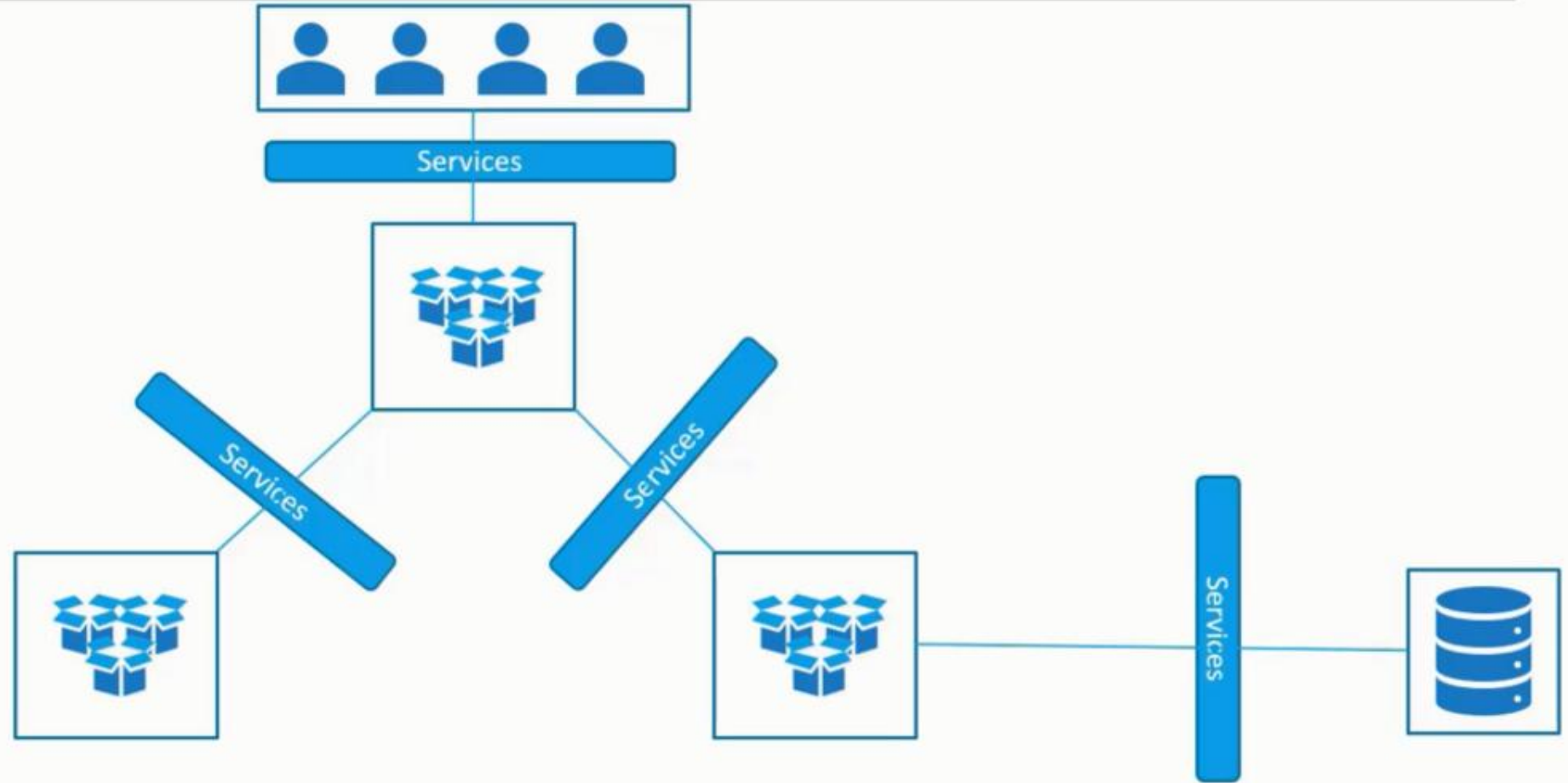
# Services

---



# Services

---





192.168.1.10

192.168.1.2

10.244.0.0

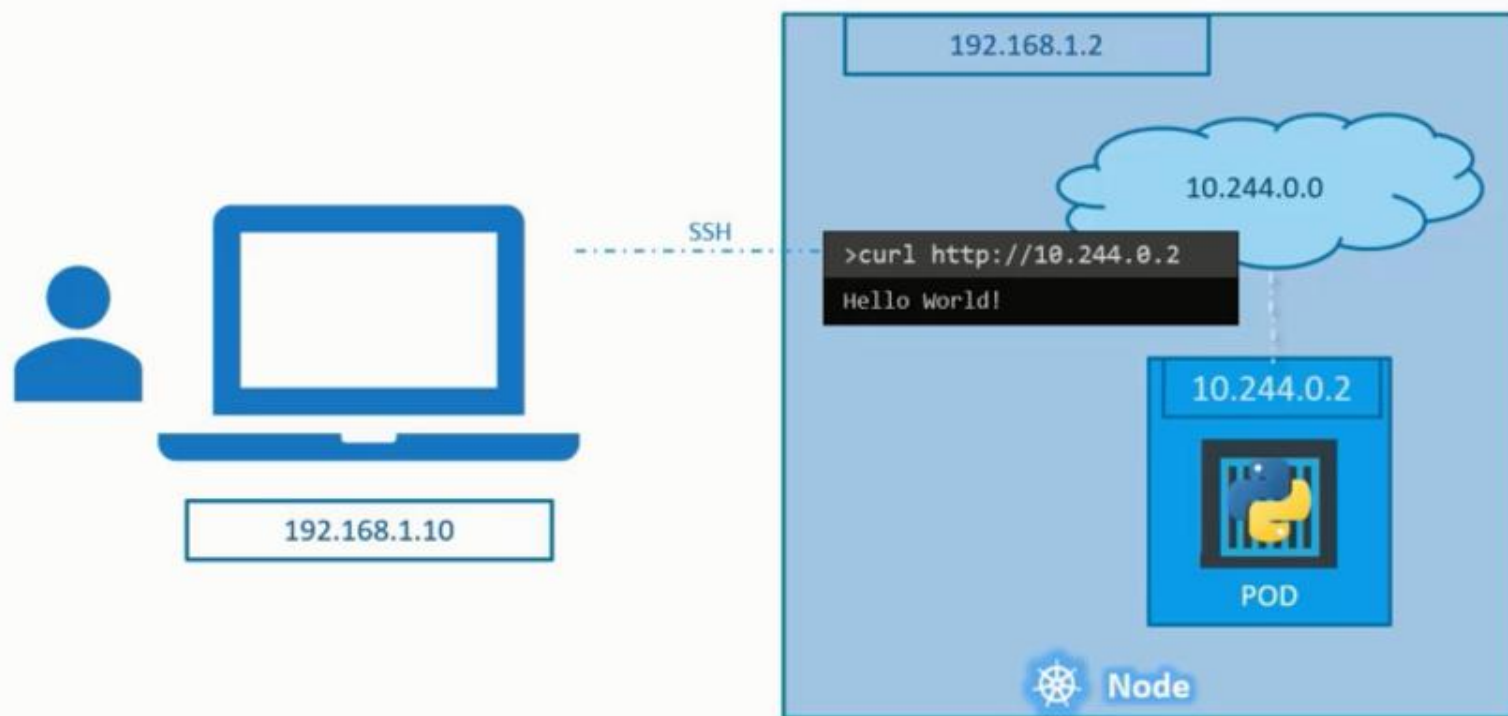
10.244.0.2

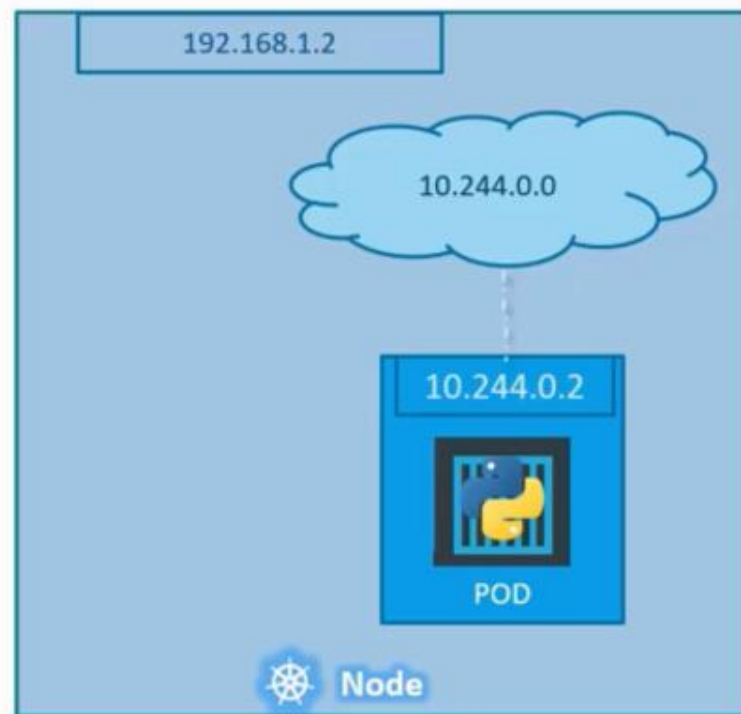
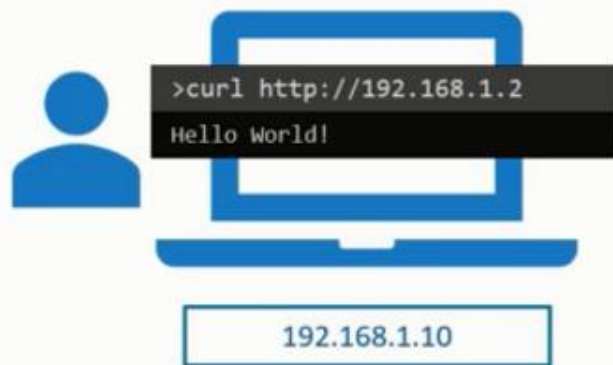


POD



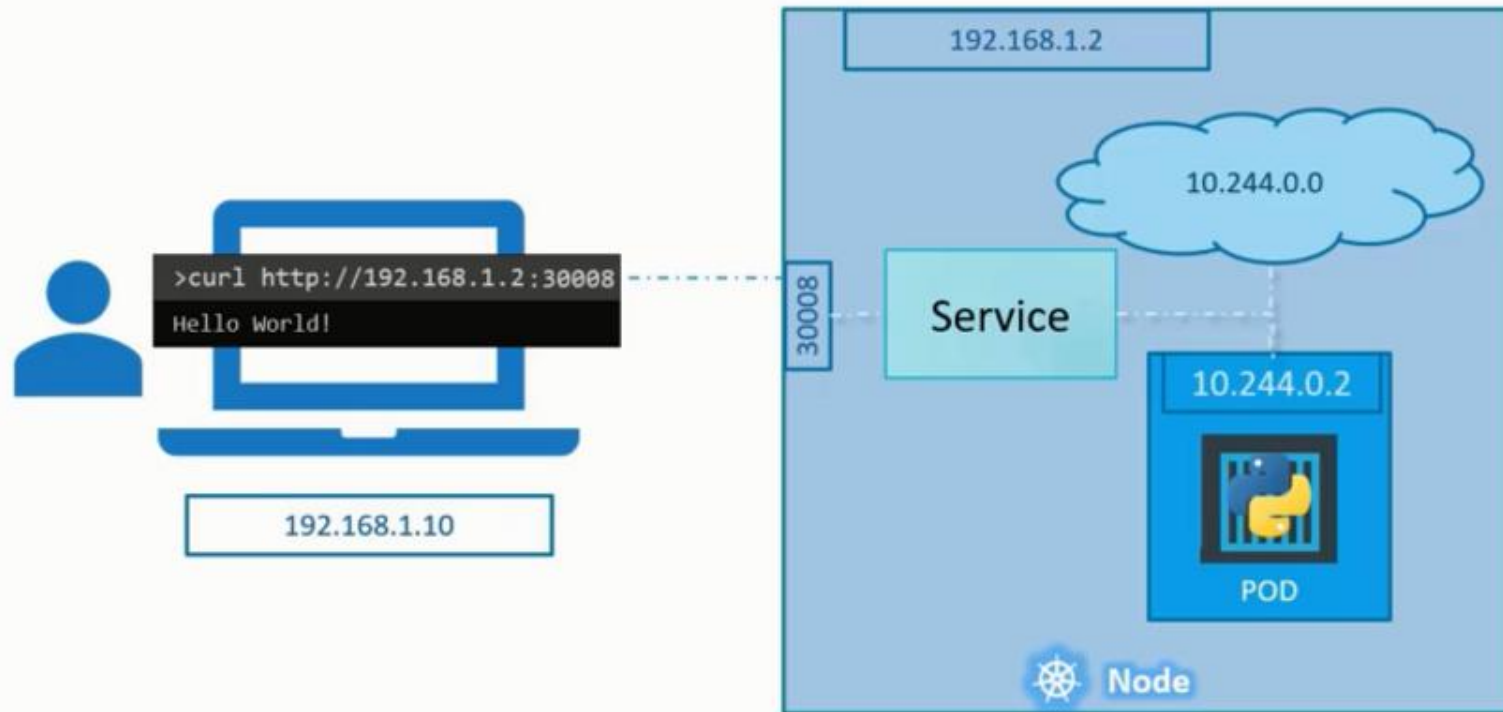
Node





# Service

---



# Services Types

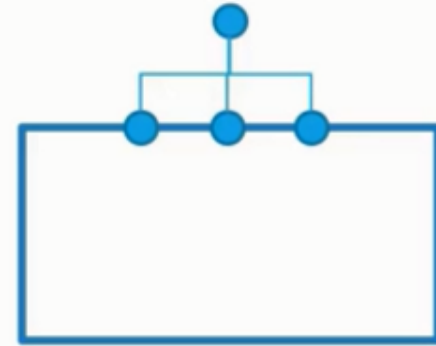
---



NodePort



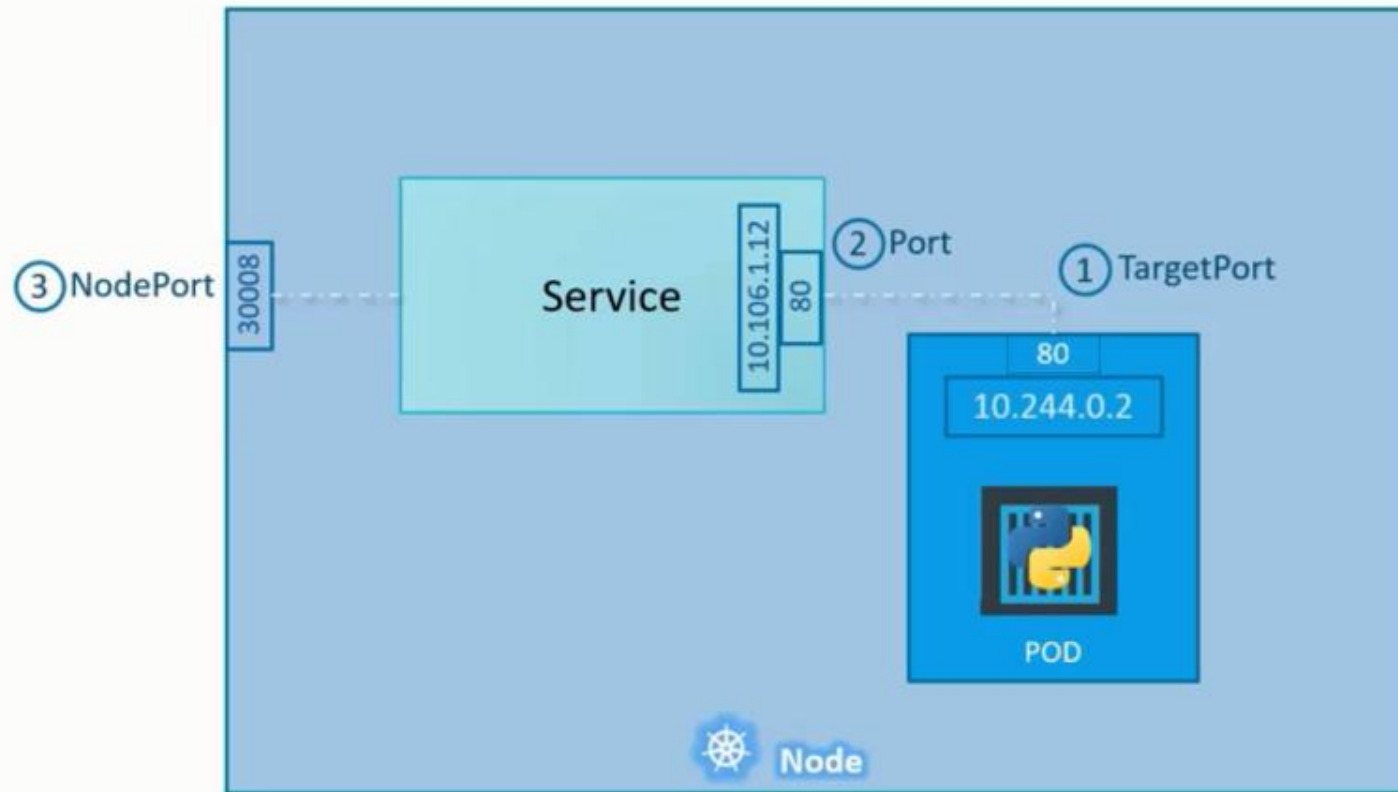
ClusterIP



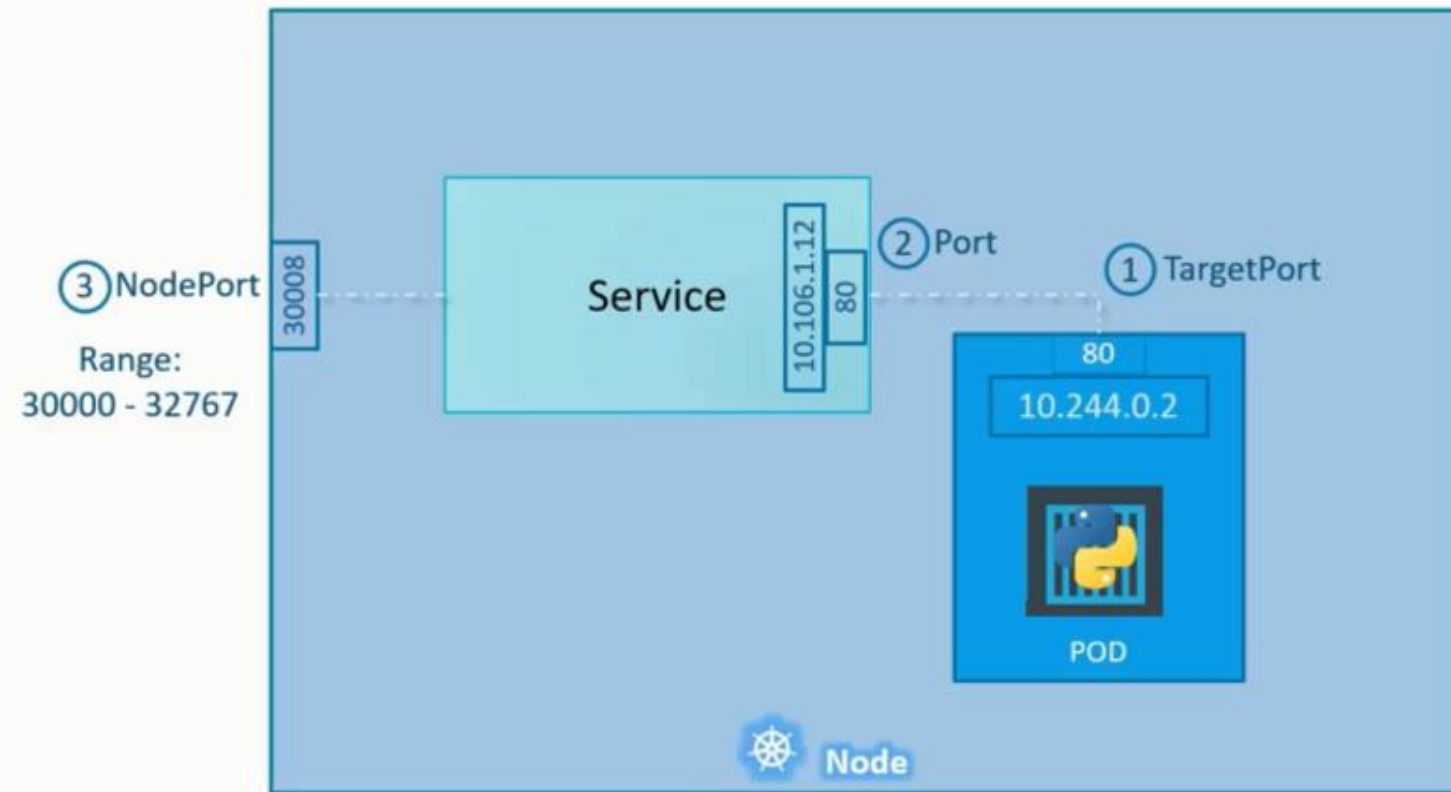
LoadBalancer

# Service - NodePort

---



# Service - NodePort



service-definition.yml

```
apiVersion:  
kind:  
metadata:  
  
spec:
```

# Service - NodePort

---

service-definition.yml

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      port: 80
      nodePort: 30008
  selector:
```

pod-definition.yml

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
    - name: nginx-container
      image: nginx
```

# Service - NodePort

---

service-definition.yml

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      port: 80
      nodePort: 30008
  selector:
    app: myapp
    type: front-end
```

pod-definition.yml

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
spec:
  containers:
    - name: nginx-container
      image: nginx
```

# Service - NodePort

---

service-definition.yml

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      port: 80
      nodePort: 30008
  selector:
    app: myapp
    type: front-end
```

```
> kubectl create -f service-definition.yml
```

```
service "myapp-service" created
```

# Service - NodePort

---

service-definition.yml

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      port: 80
      nodePort: 30008
  selector:
    app: myapp
    type: front-end
```

```
> kubectl create -f service-definition.yml
```

```
service "myapp-service" created
```

```
> kubectl get services
```

| NAME          | TYPE      | CLUSTER-IP     | EXTERNAL-IP | PORT(S)      | AGE |
|---------------|-----------|----------------|-------------|--------------|-----|
| kubernetes    | ClusterIP | 10.96.0.1      | <none>      | 443/TCP      | 16d |
| myapp-service | NodePort  | 10.106.127.123 | <none>      | 80:30008/TCP | 5m  |

# Service - NodePort

service-definition.yml

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      port: 80
      nodePort: 30008
  selector:
    app: myapp
    type: front-end
```

```
> kubectl create -f service-definition.yml
```

```
service "myapp-service" created
```

```
> kubectl get services
```

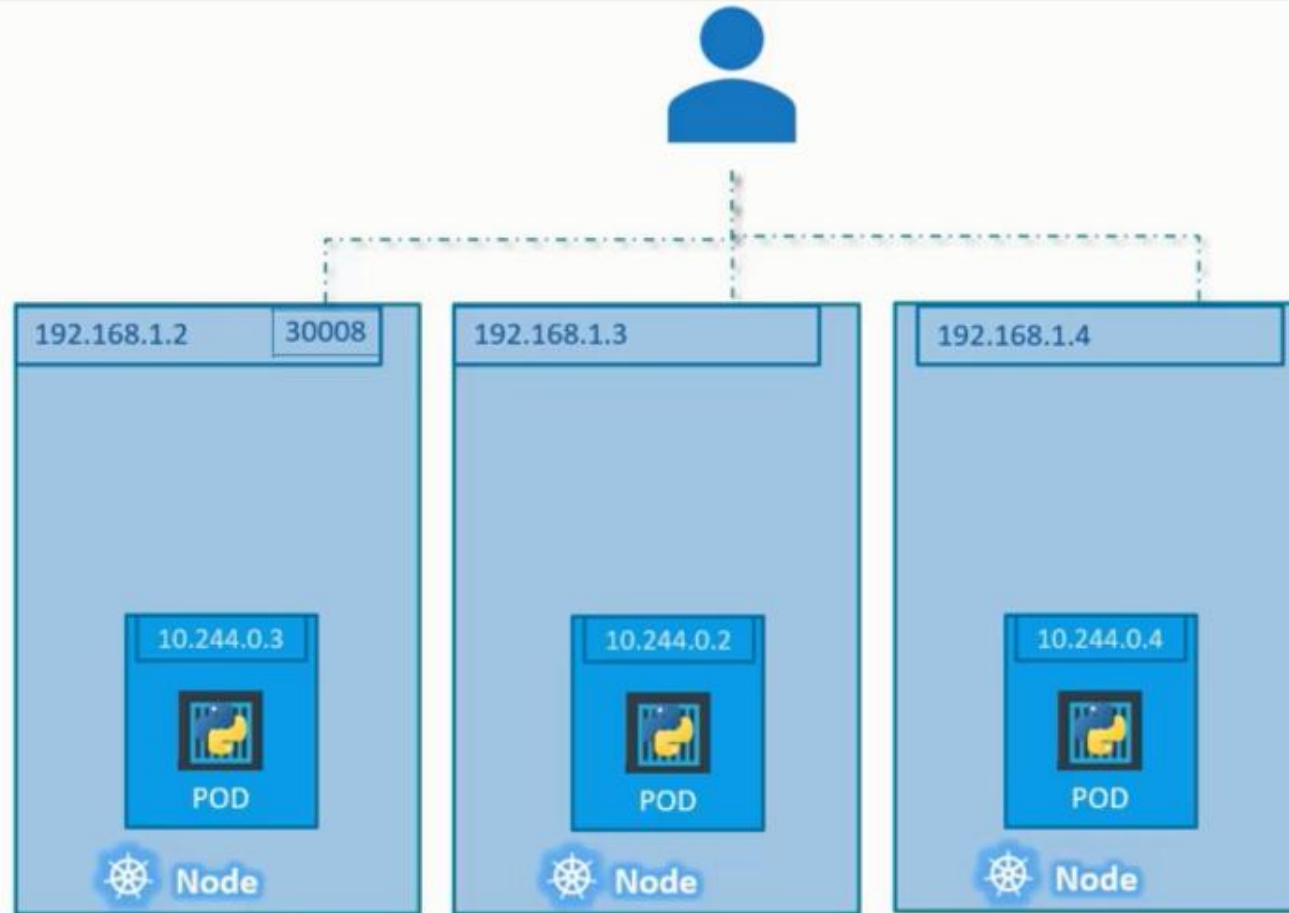
| NAME          | TYPE      | CLUSTER-IP     | EXTERNAL-IP | PORT(S)      | AGE |
|---------------|-----------|----------------|-------------|--------------|-----|
| kubernetes    | ClusterIP | 10.96.0.1      | <none>      | 443/TCP      | 16d |
| myapp-service | NodePort  | 10.106.127.123 | <none>      | 80:30008/TCP | 5m  |

```
> curl http://192.168.1.2:30008
```

```
<html>
<head>
<title>Welcome to nginx!</title>
<style>
  body {
    width: 35em;
    margin: 0 auto;
    font-family: Tahoma, Verdana, Arial, sans-serif;
  }
</style>
</head>
<body>
```

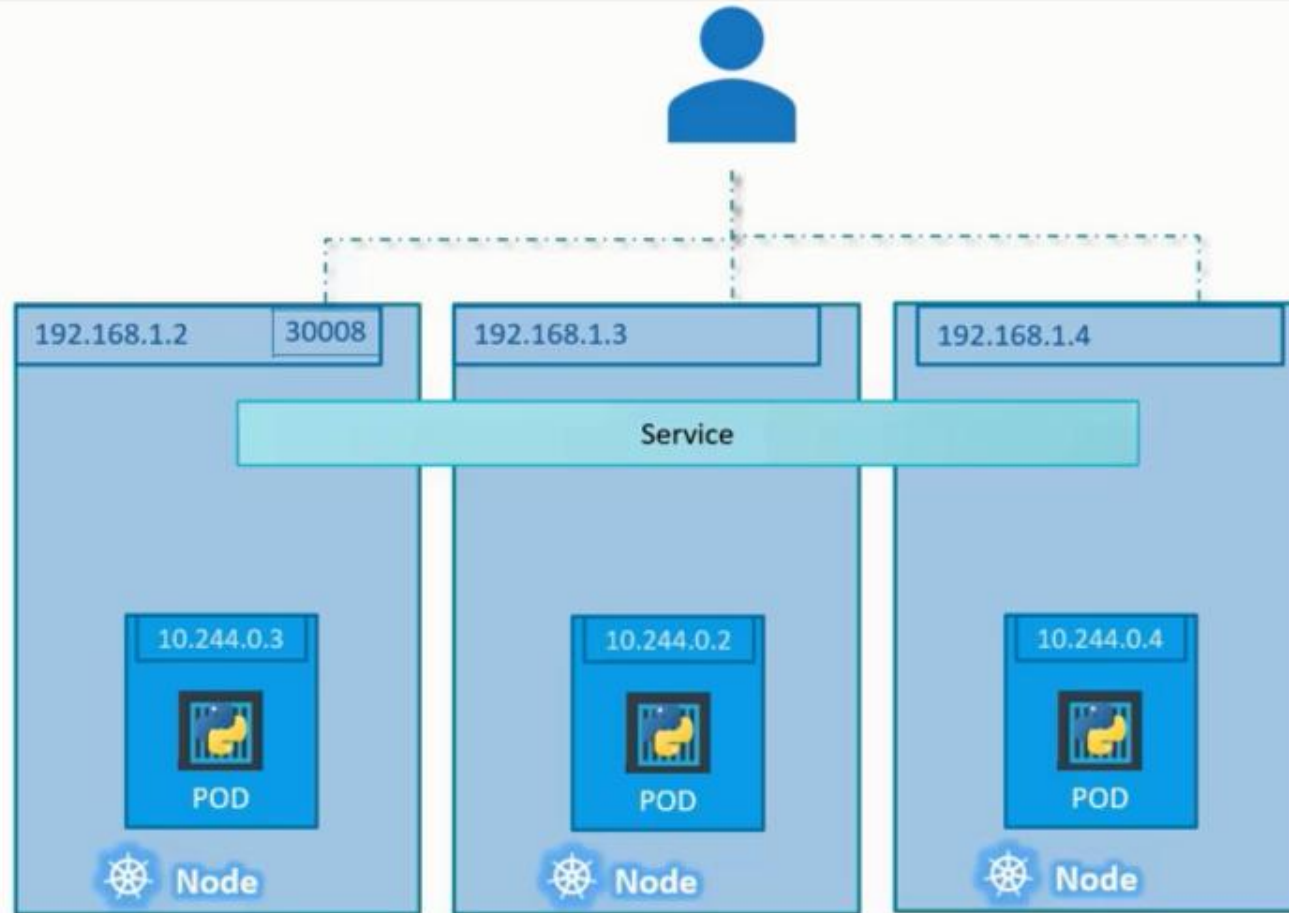
# Service - NodePort

---

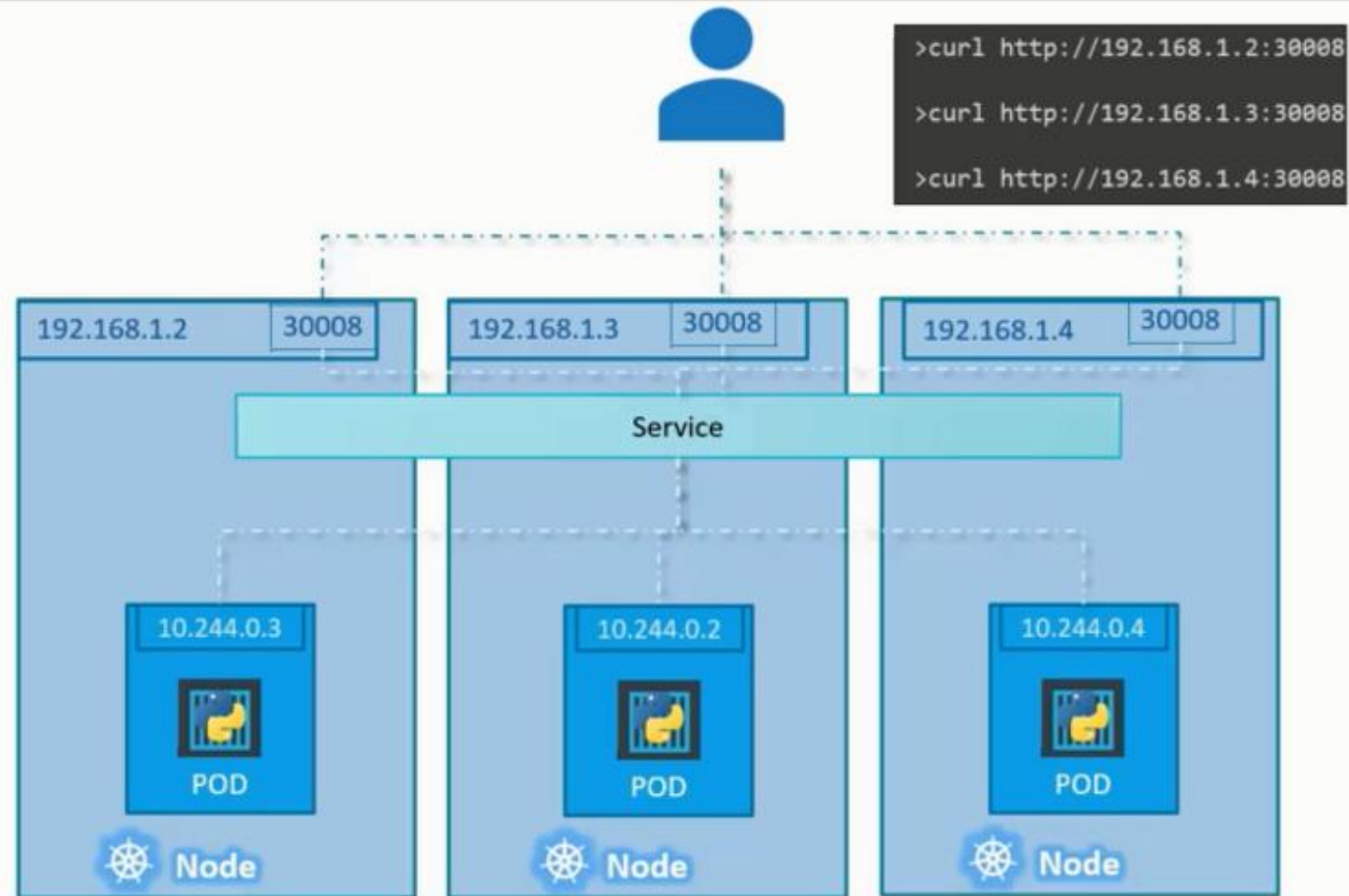


# Service - NodePort

---



# Service - NodePort

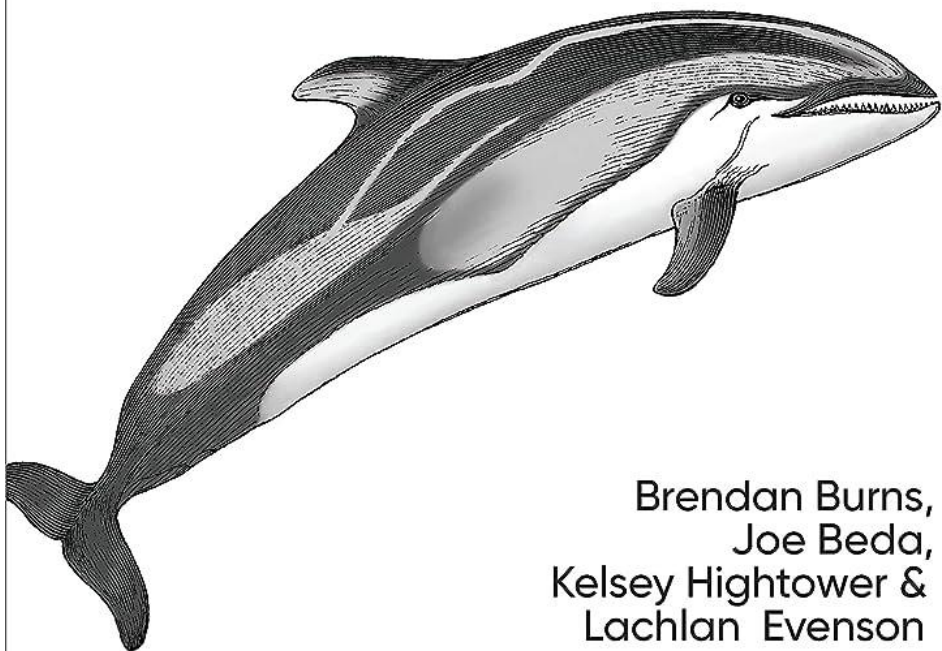


O'REILLY®

Third  
Edition

# Kubernetes Up & Running

Dive into the Future of Infrastructure



Brendan Burns,  
Joe Beda,  
Kelsey Hightower &  
Lachlan Evenson

## Table of Contents

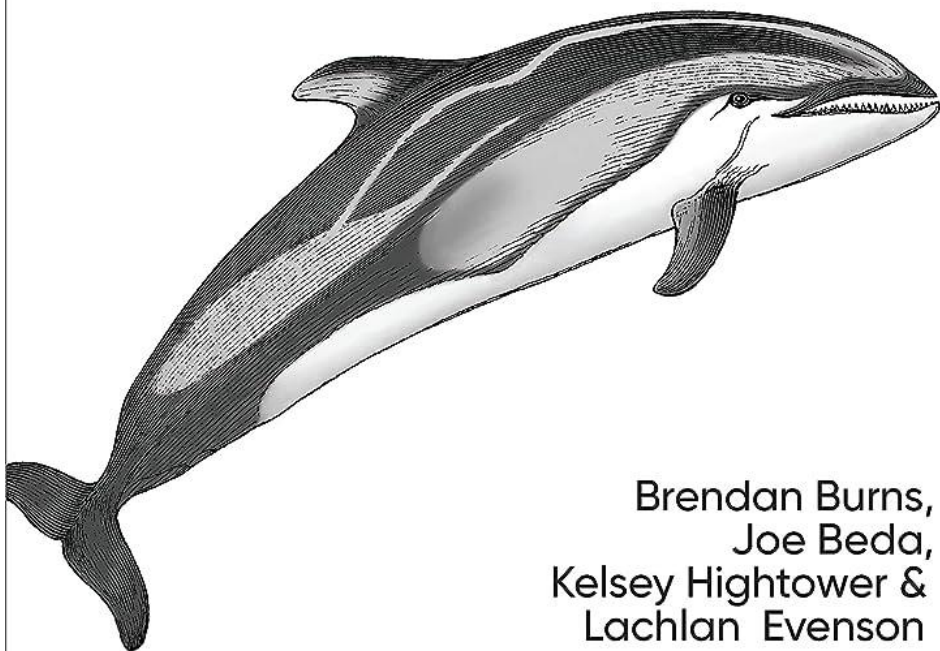
|                                                         |           |
|---------------------------------------------------------|-----------|
| Preface.....                                            | xiii      |
| <b>1. Introduction.....</b>                             | <b>1</b>  |
| Velocity.....                                           | 2         |
| The Value of Immutability.....                          | 3         |
| Declarative Configuration.....                          | 4         |
| Self-Healing Systems.....                               | 5         |
| Scaling Your Service and Your Teams.....                | 5         |
| Decoupling.....                                         | 6         |
| Easy Scaling for Applications and Clusters.....         | 6         |
| Scaling Development Teams with Microservices.....       | 7         |
| Separation of Concerns for Consistency and Scaling..... | 8         |
| Abstracting Your Infrastructure.....                    | 9         |
| Efficiency.....                                         | 10        |
| Summary.....                                            | 11        |
| <b>2. Creating and Running Containers.....</b>          | <b>13</b> |
| Container Images.....                                   | 14        |
| The Docker Image Format.....                            | 15        |
| Building Application Images with Docker.....            | 16        |
| Dockerfiles.....                                        | 16        |
| Optimizing Image Sizes.....                             | 18        |
| Image Security.....                                     | 19        |
| Multistage Image Builds.....                            | 20        |
| Storing Images in a Remote Registry.....                | 22        |
| The Docker Container Runtime.....                       | 23        |
| Running Containers with Docker.....                     | 23        |
| Exploring the kuard Application.....                    | 23        |

O'REILLY®

Third  
Edition

# Kubernetes Up & Running

Dive into the Future of Infrastructure



Brendan Burns,  
Joe Beda,  
Kelsey Hightower &  
Lachlan Evenson

## Table of Contents

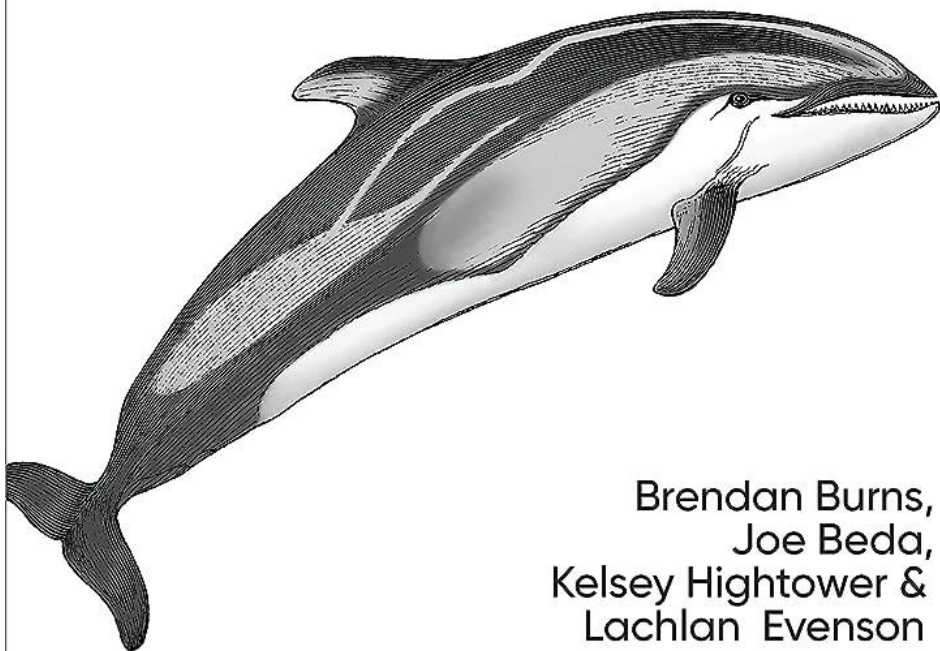
|                                                         |           |
|---------------------------------------------------------|-----------|
| Preface.....                                            | xiii      |
| <b>1. Introduction.....</b>                             | <b>1</b>  |
| Velocity.....                                           | 2         |
| The Value of Immutability.....                          | 3         |
| Declarative Configuration.....                          | 4         |
| Self-Healing Systems.....                               | 5         |
| Scaling Your Service and Your Teams.....                | 5         |
| Decoupling.....                                         | 6         |
| Easy Scaling for Applications and Clusters.....         | 6         |
| Scaling Development Teams with Microservices.....       | 7         |
| Separation of Concerns for Consistency and Scaling..... | 8         |
| Abstracting Your Infrastructure.....                    | 9         |
| Efficiency.....                                         | 10        |
| Summary.....                                            | 11        |
| <b>2. Creating and Running Containers.....</b>          | <b>13</b> |
| Container Images.....                                   | 14        |
| The Docker Image Format.....                            | 15        |
| Building Application Images with Docker.....            | 16        |
| Dockerfiles.....                                        | 16        |
| Optimizing Image Sizes.....                             | 18        |
| Image Security.....                                     | 19        |
| Multistage Image Builds.....                            | 20        |
| Storing Images in a Remote Registry.....                | 22        |
| The Docker Container Runtime.....                       | 23        |
| Running Containers with Docker.....                     | 23        |
| Exploring the kuard Application.....                    | 23        |

O'REILLY®

Third  
Edition

# Kubernetes Up & Running

Dive into the Future of Infrastructure



Brendan Burns,  
Joe Beda,  
Kelsey Hightower &  
Lachlan Evenson

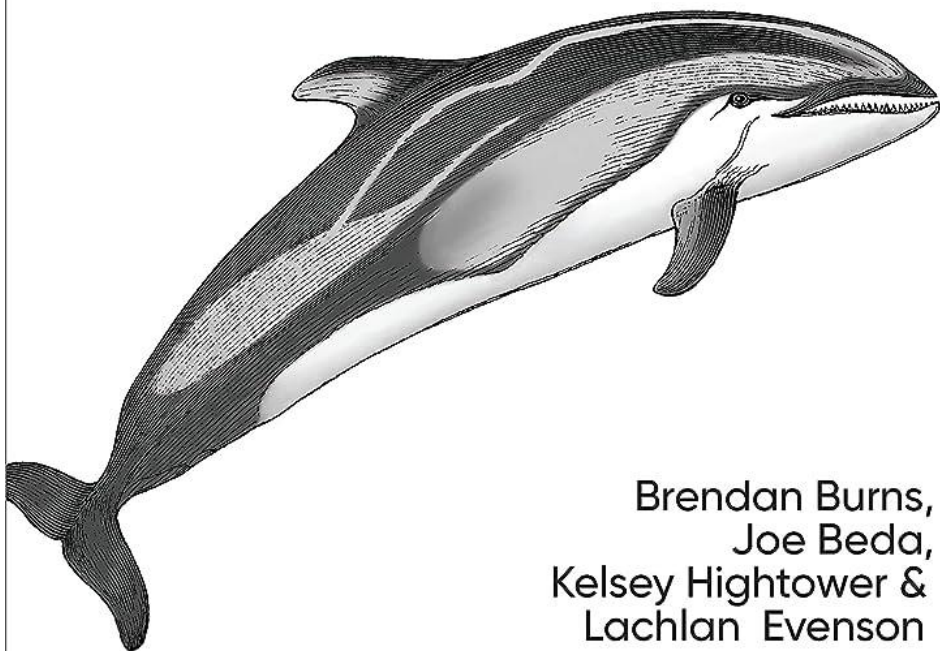
|                                                       |           |
|-------------------------------------------------------|-----------|
| Limiting Resource Usage                               | 24        |
| Cleanup                                               | 24        |
| Summary                                               | 25        |
| <b>3. Deploying a Kubernetes Cluster.....</b>         | <b>27</b> |
| Installing Kubernetes on a Public Cloud Provider      | 28        |
| Google Kubernetes Engine                              | 28        |
| Installing Kubernetes with Azure Kubernetes Service   | 28        |
| Installing Kubernetes on Amazon Web Services          | 29        |
| Installing Kubernetes Locally Using minikube          | 29        |
| Running Kubernetes in Docker                          | 30        |
| Running Kubernetes on Raspberry Pi                    | 31        |
| The Kubernetes Client                                 | 31        |
| Checking Cluster Status                               | 31        |
| Listing Kubernetes Worker Nodes                       | 32        |
| Cluster Components                                    | 34        |
| Kubernetes Proxy                                      | 34        |
| Kubernetes DNS                                        | 34        |
| Kubernetes UI                                         | 35        |
| Summary                                               | 36        |
| <b>4. Common kubectl Commands.....</b>                | <b>37</b> |
| Namespaces                                            | 37        |
| Contexts                                              | 37        |
| Viewing Kubernetes API Objects                        | 38        |
| Creating, Updating, and Destroying Kubernetes Objects | 39        |
| Labeling and Annotating Objects                       | 40        |
| Debugging Commands                                    | 40        |
| Command Autocompletion                                | 42        |
| Alternative Ways of Viewing Your Cluster              | 42        |
| Summary                                               | 43        |
| <b>5. Pods.....</b>                                   | <b>45</b> |
| Pods in Kubernetes                                    | 46        |
| Thinking with Pods                                    | 46        |
| The Pod Manifest                                      | 47        |
| Creating a Pod                                        | 48        |
| Creating a Pod Manifest                               | 48        |
| Running Pods                                          | 49        |
| Listing Pods                                          | 49        |
| Pod Details                                           | 50        |
| Deleting a Pod                                        | 51        |

O'REILLY®

Third  
Edition

# Kubernetes Up & Running

Dive into the Future of Infrastructure



Brendan Burns,  
Joe Beda,  
Kelsey Hightower &  
Lachlan Evenson

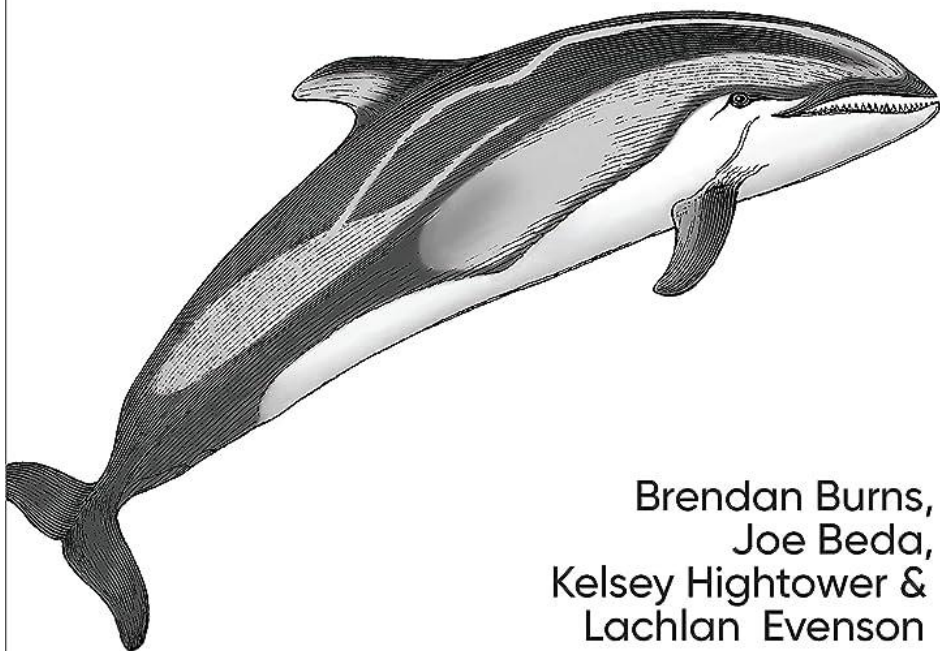
|                                               |           |
|-----------------------------------------------|-----------|
| Accessing Your Pod                            | 52        |
| Using Port Forwarding                         | 52        |
| Getting More Info with Logs                   | 52        |
| Running Commands in Your Container with exec  | 53        |
| Copying Files to and from Containers          | 53        |
| Health Checks                                 | 54        |
| Liveness Probe                                | 54        |
| Readiness Probe                               | 55        |
| Types of Health Checks                        | 56        |
| Resource Management                           | 56        |
| Resource Requests: Minimum Required Resources | 56        |
| Capping Resource Usage with Limits            | 58        |
| Persisting Data with Volumes                  | 59        |
| Using Volumes with Pods                       | 59        |
| Different Ways of Using Volumes with Pods     | 60        |
| Persisting Data Using Remote Disks            | 61        |
| Putting It All Together                       | 61        |
| Summary                                       | 63        |
| <b>6. Labels and Annotations.....</b>         | <b>65</b> |
| Labels                                        | 65        |
| Applying Labels                               | 67        |
| Modifying Labels                              | 68        |
| Label Selectors                               | 68        |
| Label Selectors in API Objects                | 70        |
| Labels in the Kubernetes Architecture         | 71        |
| Annotations                                   | 71        |
| Defining Annotations                          | 72        |
| Cleanup                                       | 73        |
| Summary                                       | 73        |
| <b>7. Service Discovery.....</b>              | <b>75</b> |
| What Is Service Discovery?                    | 75        |
| The Service Object                            | 76        |
| Service DNS                                   | 77        |
| Readiness Checks                              | 78        |
| Looking Beyond the Cluster                    | 79        |
| Cloud Integration                             | 81        |
| Advanced Details                              | 82        |
| Endpoints                                     | 82        |
| Manual Service Discovery                      | 83        |
| kube-proxy and Cluster IPs                    | 84        |

O'REILLY®

Third  
Edition

# Kubernetes Up & Running

Dive into the Future of Infrastructure



Brendan Burns,  
Joe Beda,  
Kelsey Hightower &  
Lachlan Evenson

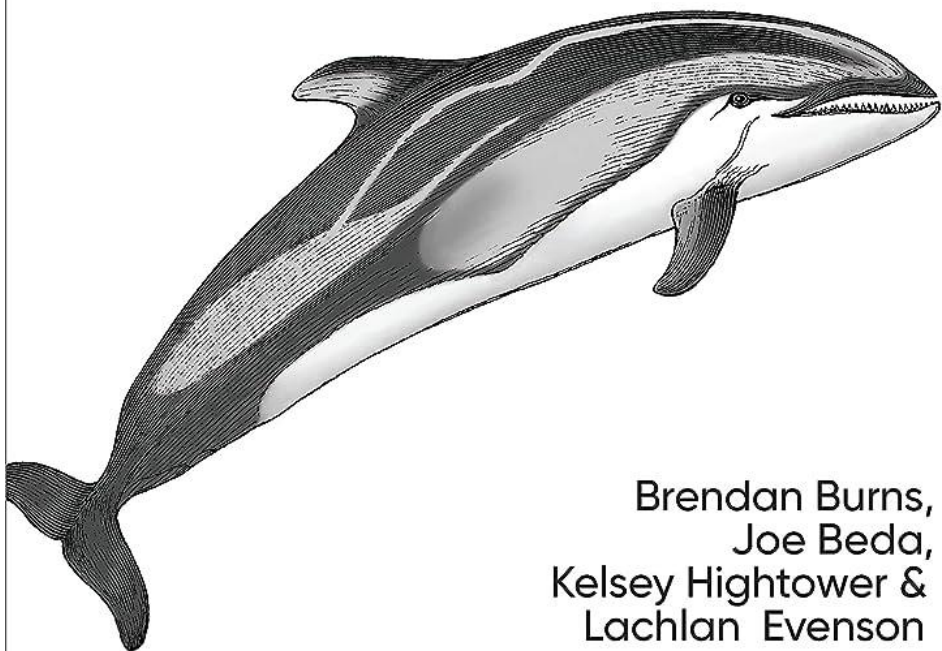
|                                                 |            |
|-------------------------------------------------|------------|
| Cluster IP Environment Variables                | 85         |
| Connecting with Other Environments              | 86         |
| Cleanup                                         | 86         |
| Summary                                         | 86         |
| <b>8. HTTP Load Balancing with Ingress.....</b> | <b>89</b>  |
| Ingress Spec Versus Ingress Controllers         | 90         |
| Installing Contour                              | 91         |
| Configuring DNS                                 | 92         |
| Configuring a Local hosts File                  | 92         |
| Using Ingress                                   | 92         |
| Simplest Usage                                  | 93         |
| Using Hostnames                                 | 94         |
| Using Paths                                     | 95         |
| Cleaning Up                                     | 96         |
| Advanced Ingress Topics and Gotchas             | 96         |
| Running Multiple Ingress Controllers            | 97         |
| Multiple Ingress Objects                        | 97         |
| Ingress and Namespaces                          | 97         |
| Path Rewriting                                  | 98         |
| Serving TLS                                     | 98         |
| Alternate Ingress Implementations               | 99         |
| The Future of Ingress                           | 100        |
| Summary                                         | 101        |
| <b>9. ReplicaSets.....</b>                      | <b>103</b> |
| Reconciliation Loops                            | 104        |
| Relating Pods and ReplicaSets                   | 104        |
| Adopting Existing Containers                    | 105        |
| Quarantining Containers                         | 105        |
| Designing with ReplicaSets                      | 105        |
| ReplicaSet Spec                                 | 106        |
| Pod Templates                                   | 106        |
| Labels                                          | 107        |
| Creating a ReplicaSet                           | 107        |
| Inspecting a ReplicaSet                         | 108        |
| Finding a ReplicaSet from a Pod                 | 108        |
| Finding a Set of Pods for a ReplicaSet          | 108        |
| Scaling ReplicaSets                             | 109        |
| Imperative Scaling with kubectl scale           | 109        |
| Declaratively Scaling with kubectl apply        | 109        |
| Autoscaling a ReplicaSet                        | 110        |

O'REILLY®

Third  
Edition

# Kubernetes Up & Running

Dive into the Future of Infrastructure



Brendan Burns,  
Joe Beda,  
Kelsey Hightower &  
Lachlan Evenson

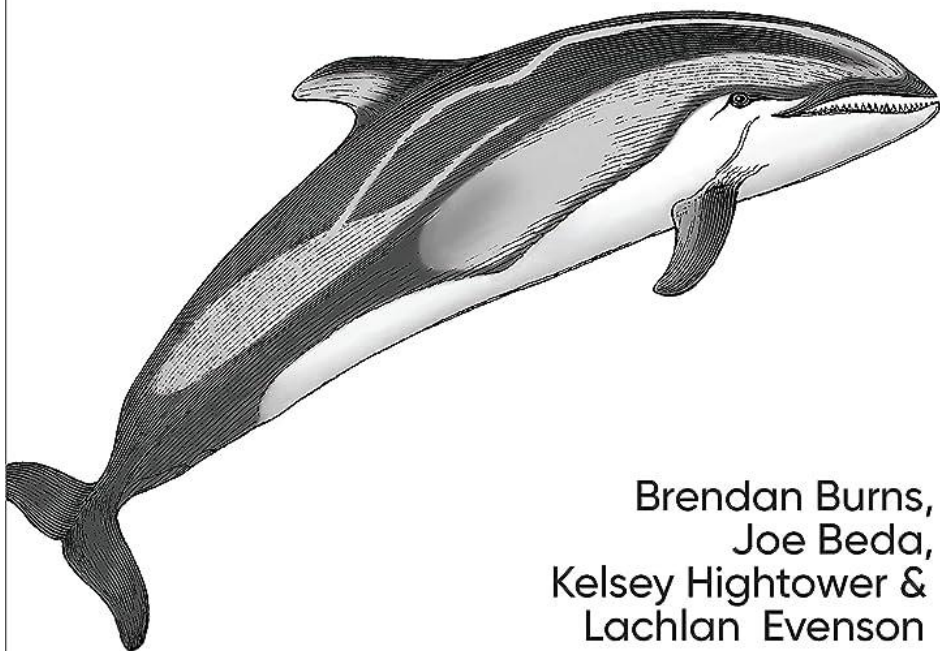
|                                           |            |
|-------------------------------------------|------------|
| Deleting ReplicaSets                      | 111        |
| Summary                                   | 112        |
| <b>10. Deployments</b>                    | <b>113</b> |
| Your First Deployment                     | 114        |
| Deployment Internals                      | 114        |
| Creating Deployments                      | 116        |
| Managing Deployments                      | 117        |
| Updating Deployments                      | 118        |
| Scaling a Deployment                      | 118        |
| Updating a Container Image                | 119        |
| Rollout History                           | 120        |
| Deployment Strategies                     | 123        |
| Recreate Strategy                         | 123        |
| RollingUpdate Strategy                    | 123        |
| Slowing Rollouts to Ensure Service Health | 126        |
| Deleting a Deployment                     | 128        |
| Monitoring a Deployment                   | 128        |
| Summary                                   | 129        |
| <b>11. DaemonSets</b>                     | <b>131</b> |
| DaemonSet Scheduler                       | 132        |
| Creating DaemonSets                       | 132        |
| Limiting DaemonSets to Specific Nodes     | 134        |
| Adding Labels to Nodes                    | 135        |
| Node Selectors                            | 135        |
| Updating a DaemonSet                      | 136        |
| Rolling Update of a DaemonSet             | 136        |
| Deleting a DaemonSet                      | 137        |
| Summary                                   | 138        |
| <b>12. Jobs</b>                           | <b>139</b> |
| The Job Object                            | 139        |
| Job Patterns                              | 140        |
| One Shot                                  | 140        |
| Parallelism                               | 144        |
| Work Queues                               | 146        |
| CronJobs                                  | 150        |
| Summary                                   | 151        |
| <b>13. ConfigMaps and Secrets</b>         | <b>153</b> |
| ConfigMaps                                | 153        |

O'REILLY®

Third  
Edition

# Kubernetes Up & Running

Dive into the Future of Infrastructure



Brendan Burns,  
Joe Beda,  
Kelsey Hightower &  
Lachlan Evenson

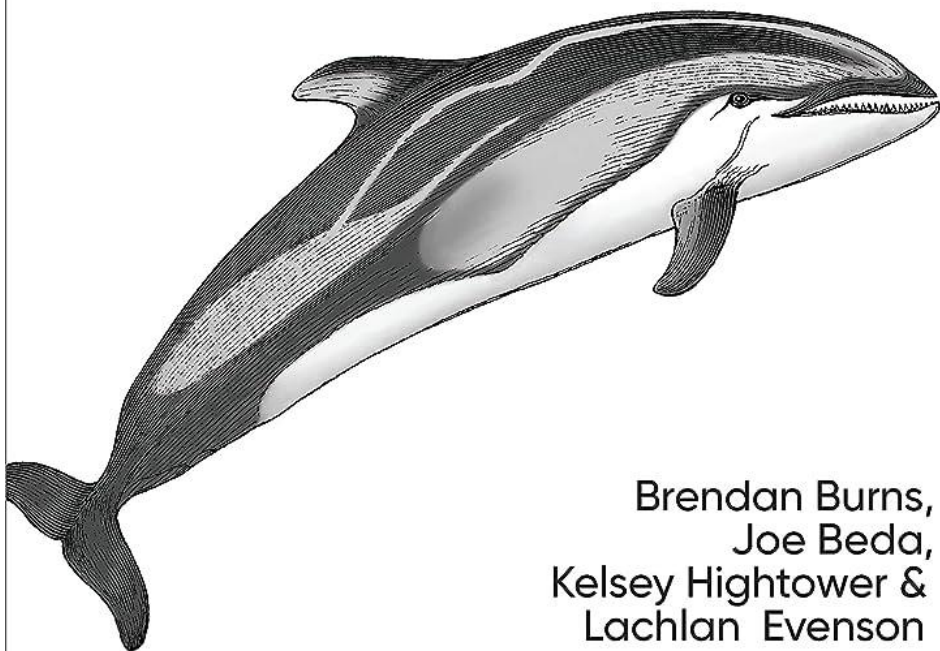
|                                                               |            |
|---------------------------------------------------------------|------------|
| Creating ConfigMaps                                           | 153        |
| Using a ConfigMap                                             | 154        |
| Secrets                                                       | 157        |
| Creating Secrets                                              | 158        |
| Consuming Secrets                                             | 159        |
| Private Docker Registries                                     | 160        |
| Naming Constraints                                            | 161        |
| Managing ConfigMaps and Secrets                               | 162        |
| Listing                                                       | 162        |
| Creating                                                      | 163        |
| Updating                                                      | 163        |
| Summary                                                       | 165        |
| <b>14. Role-Based Access Control for Kubernetes. ....</b>     | <b>167</b> |
| Role-Based Access Control                                     | 168        |
| Identity in Kubernetes                                        | 168        |
| Understanding Roles and Role Bindings                         | 169        |
| Roles and Role Bindings in Kubernetes                         | 169        |
| Techniques for Managing RBAC                                  | 172        |
| Testing Authorization with can-i                              | 172        |
| Managing RBAC in Source Control                               | 172        |
| Advanced Topics                                               | 172        |
| Aggregating ClusterRoles                                      | 173        |
| Using Groups for Bindings                                     | 173        |
| Summary                                                       | 175        |
| <b>15. Integrating Storage Solutions and Kubernetes. ....</b> | <b>177</b> |
| Importing External Services                                   | 178        |
| Services Without Selectors                                    | 179        |
| Limitations of External Services: Health Checking             | 181        |
| Running Reliable Singletons                                   | 181        |
| Running a MySQL Singleton                                     | 181        |
| Dynamic Volume Provisioning                                   | 185        |
| Kubernetes-Native Storage with StatefulSets                   | 186        |
| Properties of StatefulSets                                    | 187        |
| Manually Replicated MongoDB with StatefulSets                 | 187        |
| Automating MongoDB Cluster Creation                           | 189        |
| Persistent Volumes and StatefulSets                           | 192        |
| One Final Thing: Readiness Probes                             | 193        |
| Summary                                                       | 194        |

O'REILLY®

Third  
Edition

# Kubernetes Up & Running

Dive into the Future of Infrastructure



Brendan Burns,  
Joe Beda,  
Kelsey Hightower &  
Lachlan Evenson

|                                                                       |            |
|-----------------------------------------------------------------------|------------|
| <b>16. Extending Kubernetes.....</b>                                  | <b>195</b> |
| What It Means to Extend Kubernetes                                    | 195        |
| Points of Extensibility                                               | 196        |
| Patterns for Custom Resources                                         | 204        |
| Just Data                                                             | 204        |
| Compilers                                                             | 205        |
| Operators                                                             | 205        |
| Getting Started                                                       | 205        |
| Summary                                                               | 205        |
| <b>17. Deploying Real-World Applications.....</b>                     | <b>207</b> |
| Jupyter                                                               | 207        |
| Parse                                                                 | 209        |
| Prerequisites                                                         | 209        |
| Building the parse-server                                             | 209        |
| Deploying the parse-server                                            | 209        |
| Testing Parse                                                         | 210        |
| Ghost                                                                 | 211        |
| Configuring Ghost                                                     | 211        |
| Redis                                                                 | 214        |
| Configuring Redis                                                     | 215        |
| Creating a Redis Service                                              | 216        |
| Deploying Redis                                                       | 217        |
| Playing with Our Redis Cluster                                        | 218        |
| Summary                                                               | 219        |
| <b>18. Organizing Your Application.....</b>                           | <b>221</b> |
| Principles to Guide Us                                                | 221        |
| Filesystems as the Source of Truth                                    | 222        |
| The Role of Code Review                                               | 222        |
| Feature Gates and Guards                                              | 223        |
| Managing Your Application in Source Control                           | 224        |
| Filesystem Layout                                                     | 224        |
| Managing Periodic Versions                                            | 225        |
| Structuring Your Application for Development, Testing, and Deployment | 227        |
| Goals                                                                 | 227        |
| Progression of a Release                                              | 227        |
| Parameterizing Your Application with Templates                        | 229        |
| Parameterizing with Helm and Templates                                | 229        |
| Filesystem Layout for Parameterization                                | 230        |
| Deploying Your Application Around the World                           | 230        |
| Architectures for Worldwide Deployment                                | 230        |

In the next sessions, we'll see the  
practical stuffs of



**kubernetes**